Characterizing the Feasible Region for Self-Modifying Substrates in Interactive Environments

Hyun Jun Han

2026-03-19

The shared artifact. Birth writes theory. Experiment writes results. Compress edits both.

Abstract

We formalize six rules (R1-R6) for recursive self-improvement as mathematical conditions on a state-update function $f : S \times X \to S$ and derive necessary properties of any system satisfying all six simultaneously. From 720+ experiments across 12 architecture families on ARC-AGI-3 interactive games, we extract 26 constraints and prove: (1) no satisfying system has a fixed point — self-modification is necessary, not optional; (2) in finite environments, the system must process its own internal state to maintain irredundant growth (the self-observation requirement); (3) the feasible region is non-empty for Level 1 navigation but currently unoccupied for the full constraint set including R3 (self-modification of operations). Whether a substrate exists inside all six walls remains open. The contribution is the walls themselves.

1. Introduction

720+ experiments across 12 architecture families (codebook/LVQ, LSH, L2 k-means, reservoir, Hebbian, Recode/self-refining LSH, graph, SplitTree, Absorb, connected-component, Bloom filter, CA) tested substrates for navigation and classification on ARC-AGI-3 interactive games and a cross-domain chain benchmark (CIFAR-100 → ARC-AGI-3 → CIFAR-100). All experiments used the same evaluation framework (R1-R6) and constraint map.

The experiments carved a feasible region — the set of systems that satisfy all constraints simultaneously. This paper formalizes the constraints mathematically, derives necessary properties of the feasible region, and states honestly what is proven vs conjectured vs open.

The paper is valid regardless of outcome. If a substrate satisfying all six rules is found, the feasible region is non-empty and the characterization guided the search. If no such substrate exists, the characterization identifies which constraints are mutually exclusive — also a contribution. We report whichever is true at time of writing.

2. Related Work

2.1 Self-referential self-improvement

Schmidhuber's Gödel Machine (2003) is the closest formal framework: a self-referential system that rewrites its own code when it can prove the rewrite is useful. Key differences from our framework: (1) Gödel machines require a utility function (external objective — our R1 prohibits this), (2) provable self-improvement is limited by Gödel's incompleteness theorem, (3) the Gödel machine framework doesn't address exploration saturation in finite environments.

2.2 Intrinsic motivation and curiosity-driven exploration

Pathak et al. (2017) formalize curiosity as prediction error in a learned feature space. Count-based exploration (Bellemare et al., 2016) uses state visitation counts. Both address exploration in sparse-reward environments. Key difference: these methods add intrinsic REWARD signals, which function as objectives. Our R1 requires no objectives. Our derivation shows self-observation is required by the constraint system itself, not as a reward mechanism.

2.3 Autopoiesis

Maturana & Varela (1972) define autopoietic systems as networks that produce the components that produce the network. Organizationally closed. Related to our fixed-point conjecture (Section 4.4). Key difference: autopoiesis maintains structure (homeostasis); our U17 requires unbounded growth. Our system is autopoietic + growth.

2.4 Growing neural gas and self-organizing maps

Fritzke (1995) GNG, Kohonen (1988) SOM/LVQ. The codebook substrate is LVQ + growing codebook. Well-characterized in the literature. Our contribution is not the architecture — it's the constraint map extracted from systematic testing.

2.5 Graph-based exploration for ARC-AGI-3

Rudakov et al. (2025) independently developed a training-free graph-based exploration method for ARC-AGI-3, ranking 3rd on the private leaderboard (30/52 levels across 6 games). Their method uses connected-component segmentation for frame encoding, hierarchical action prioritization (5 tiers by visual salience), and BFS path planning to frontier states. Key parallels: (1) graph-structured state tracking with hash-based node identity — equivalent to our $\pi : X \to N$; (2) click-space as expanded action space (4,096 actions for click games) — same finding as our Steps 503/505; (3) frontier-directed exploration — this IS the purposeful exploration (I6/I9) we identified as the Level 2 bottleneck, and they implemented it successfully. Key differences: their method is training-free (no learning, no self-modification) and assumes deterministic, fully observable environments. It satisfies R1 but not R3 — the graph accumulates but the exploration strategy is fixed. Their limitation ("exhaustive exploration becomes computationally intractable" at higher levels) aligns with our Theorem 2: without self-observation, exploration saturates.

2.6 Object-centric Bayesian game learning

Heins et al. (2025, "AXIOM") combine object-centric scene decomposition with online Bayesian model expansion and reduction, learning games in ~10K steps without backpropagation. Key mechanisms: (1) Slot Mixture Model segments frames into objects with position/color/shape — far richer than spatial averaging; (2) online expansion grows mixture components when new data doesn't fit (equivalent to spawn-on-novelty); (3) Bayesian Model Reduction (BMR) merges redundant components every 500 frames — consolidation that prevents centroid explosion while preserving information. AXIOM outperforms DreamerV3 and BBF on 10/10 games at 10K steps. Key differences from our framework: AXIOM uses reward signals for planning (violates R1), operations are fixed (violates R3), and BMR requires a prescribed merge criterion (frozen frame). However, BMR addresses our U3-vs-R6 tension directly: U3 says never delete, R6 says no redundancy, BMR resolves this by merging (not deleting) redundant structure.

2.7 Continual learning and catastrophic forgetting

McCloskey & Cohen (1989) identified catastrophic forgetting — neural networks lose previous knowledge when learning new tasks. The continual learning literature (survey: van de Ven & Tolias, 2024, arXiv:2403.05175) identifies six main approaches: replay, parameter regularization, functional regularization, optimization-based, context-dependent processing, and template-based classification. All assume a neural network with backpropagation (violates R1/R2). Our chain benchmark (Section 5.4) tests forgetting WITHOUT any mitigation mechanism — the substrate must naturally resist forgetting through its dynamics alone (U3: zero forgetting by construction, not by regularization).

2.8 Stigmergy and ant colony optimization

Stigmergy (Grassé, 1959) is indirect communication through environment modification — agents mark the environment, and those marks guide future behavior. Ant colony optimization (Dorigo et al., 2000) applies this to search: pheromone trails reinforce successful paths. Our substrate is a stigmergic system with **anti-pheromone**: edge counts mark visited transitions, and argmin follows the LEAST marked path. The graph IS the environment modification; the frozen frame IS the response rule. R3, in stigmergic terms, asks: can the agent modify its own pheromone response? ACO agents cannot — their response to pheromone is fixed. This is . An agent that learns to weight pheromone differently based on context would be _. An agent that modifies the rule for how it modifies pheromone would be _F.

Termite mound construction (Grassé, 1959; PNAS 2019, 2021) extends the stigmergy analogy. Termites build complex adaptive architecture without blueprints, centralized control, or global observation. Individual termites respond only to LOCAL stimuli — pheromone concentration, surface curvature, temperature gradients. The mound emerges. Crucially, termites solve our Proposition 13 problem (introspection foresight) by NOT introspecting: no termite models the global mound structure. They use local gradients as implicit predictions — pheromone concentration IS the prediction that "this location was useful."

Connection to Proposition 13: Our eigenform (Steps 620-629) tried GLOBAL self-observation (read the graph's statistics). Inert — because global self-observation computes functions of visited states, not unvisited ones. Argmin IS local gradient-following (least-visited action from current cell)

and WORKS for L1. The termite lesson for L2: the gradient must point toward the FRONTIER, not toward less-visited territory within the attractor. Not "which action has fewest visits from this cell?" but "which action leads toward cells with the most unknown successors?" Foresight via local gradient, not via world model. This is a degree of freedom (DoF 8) concretized by biological precedent.

2.9 Somatic hypermutation and self-modifying comparison

The adaptive immune system is the clearest biological example of a system that discovers its own comparison operation. B cell antibodies bind antigens through variable regions — the binding site IS the "compare" function. Somatic hypermutation (SHM; Neuberger & Milstein, 1995) introduces point mutations into the variable region during germinal center reactions, modifying the comparison function itself. Clonal selection retains high-affinity mutants (the "select" operation). Memory B cells persist indefinitely (the "store" operation, satisfying U3). Recent work (Tan et al., 2025, *Immunity*) demonstrates that non-cognate B cells can generate *de novo* antigen recognition — the system discovers entirely new binding specificities through mutation, not just refining existing ones.

Connection to Proposition 14: The immune system achieves ℓ_π (self-modification of the comparison function) via population-level mutation + selection. The frozen frame is the mutation mechanism itself (AID enzyme, activation-induced cytidine deaminase). The abstract operations {compare, select, store, modify} are entailed by the need to respond to novel pathogens. The concrete implementation (AID, VDJ recombination) is not — it evolved. This is exactly the abstract-vs-concrete gap: the WHAT is necessary, the HOW is frozen.

Connection to Step 607 (GRN, KILLED): Our population-level R3 attempt failed because encodings were not diverse enough (Cov(w,z)=0 in the Price equation). The immune system succeeds with ~10^9 B cells and targeted mutagenesis (SHM only mutates variable regions, not framework regions). The lesson: population R3 requires (1) massive parallelism, (2) targeted variation (mutate only the comparison operation, not the infrastructure), and (3) genuine selection pressure (antigen binding is binary and environmental — R5-compliant).

Germinal center regulation (Nakagawa et al., 2025, *Nature*): B cells modify their own mutation RATE to preserve high-affinity receptors. This is meta-self-modification — the system modifies how fast it modifies itself. In our hierarchy: not just ℓ_π (modify the comparison) but approaching ℓ_F (modify the modification rule). Whether this constitutes genuine ℓ_F depends on whether the rate-modification mechanism is itself fixed (frozen frame) or adaptive.

2.10 Retinal contrast adaptation and selective -refinement

Retinal ganglion cells (RGCs) solve Proposition 15's open question — targeted observation-mapping refinement without foreknowledge of what's important — via local contrast adaptation. Each RGC independently adjusts its receptive field based on recent stimulus statistics (Baccus & Meister, 2002, *Neuron*; Gollisch & Meister, 2010). Two mechanisms operate at the same spatial location: **adaptation** (reduce sensitivity at familiar stimuli, via bipolar cell synaptic depression) and **sensitization** (increase sensitivity at novel/variable stimuli, via amacrine cell inhibition — Kastner & Bhatt, 2018, PMC). Dynamic receptive fields are "flexible and fluid" (Rivlin-Etzion et al., 2018), adjusting shape based on stimulus content.

Connection to Proposition 15: The substrate's bottleneck is hash resolution at critical cells (k_{n^*}, the number of aliased hidden states). Step 669 attempted local variance-driven refinement — running variance per cell, fine hash of high-variance pixels. Result: MARGINAL 5/10. Seeds with few high-variance cells (91) solved 285x faster; seeds with many (286) lost L1. The mechanism WORKS locally but fails to SELECT which cells to refine.

The retina's solution has three features the substrate lacks: 1. **Cell-type diversity** — different RGC types have different adaptation profiles (Kastner & Baccus, 2017). Multiple "hash functions" at different resolutions operate SIMULTANEOUSLY, not one function that switches. The substrate analog: maintain LSH at multiple k values (e.g., k=12 for navigation, k=20 for refinement at variable cells) and let local statistics determine which resolution to read from. 2. **Spatially segregated adaptation/sensitization** — adaptation at the center, sensitization at the surround (Baccus & Meister, 2002). The cell refines its resolution at the BOUNDARY of its receptive field, not uniformly. For the substrate: refine hash at cells where transition outcomes are MOST variable, not where pixel variance is highest. 3. **No foreknowledge** — each RGC adapts based on its own stimulus history. No global controller identifies "critical cells." The retina doesn't know which locations will be important — it maintains resolution everywhere and INCREASES it where variability is detected. This is exactly the R1-compliant π-refinement Proposition 15 requires.

Degree of freedom concretized: Recode (Step 542) refines π from transition statistics but treats all cells equally. The retinal insight suggests SELECTIVE refinement: cells with high transition variability get finer resolution (sensitization), cells with low variability maintain coarse resolution (adaptation). The intervention is determined by local cell statistics, not global analysis — R1-compliant by construction.

2.11 Renormalization group and the Level 2 phase transition

The renormalization group (RG; Kadanoff 1966, Wilson 1971) formalizes how physical systems behave across scales. The core operation — "block spinning" — averages microscopic variables into blocks and asks: does the effective Hamiltonian at the coarser scale have the same structure? At critical points, the answer is yes (scale invariance), and the system flows along a universality class determined by symmetries, not by microscopic details.

Connection to encoding: Our avgpool_{16}: $64 \times 64 \to 16 \times 16$ IS Kadanoff blocking — averaging pixels in 4×4 blocks. U16 (centering is load-bearing) says this specific coarse-graining preserves the information needed for navigation. The "right" blocking is one that preserves the task-relevant symmetries while discarding irrelevant microscopic detail. Step 674 (transition-triggered dual-hash, k=12 coarse + k=20 fine) is a multi-scale RG: the coarse scale identifies WHERE finer resolution is needed (aliased cells), and the fine scale resolves the hidden-state ambiguity there.

Connection to the L2 wall: In RG, a phase transition occurs when the effective Hamiltonian at a new scale has DIFFERENT relevant operators — the symmetry class changes. L1 $\to$ L2 is precisely this: L1 requires resolving ≤ 4 hidden states at the exit cell (the conjunction condition). L2 introduces energy sprites, extended state variables, and multi-step temporal dependencies — qualitatively different relevant operators. The L2 wall is not a refinement problem (making L1 finer does not reach L2, confirmed Steps 486-492, 686). It is a relevance problem: L2 has different relevant operators than L1, analogous to crossing a phase boundary in RG flow where the same coarse-graining produces a qualitatively different effective theory.

Implication for R3: RG shows that effective theories at different scales can be expressed using the SAME mathematical structure (Hamiltonians) with different parameters. The substrate analog: compare-select-store may be expressively sufficient for L2 if the state space can encode scale-dependent operators — operations that change their behavior depending on which "level" of the game's hidden state the agent is in. This is the $\ell_1 \to \ell_F$ bridge: the interpreter stays fixed, but the state encodes WHICH comparison to apply at WHICH scale, and the system discovers this mapping from interaction. The RG framing sharpens Q21: not "can compare-select-store encode arbitrary modifications?" but "can compare-select-store encode the RELEVANT operators at each scale?"

3. Formal Framework

On the status of R1-R6: The six rules began as philosophical commitments. The experiments validated them — each rule is justified by what fails when it is violated:

- **R1 (no external objectives):** Every targeted exploration strategy failed (Steps 478-481: novelty 1/10, prediction-error 0/10, UCB1 neutral). External objectives create exploitable structure that noisy environments corrupt. Argmin survives because it has no target to corrupt.
- **R2 (adaptation from computation):** Algorithm invariance across 4 families (codebook, LSH, Hebbian, Markov — Steps 524-525). The same argmin algorithm emerges regardless of representation. Adaptation IS the computation, not a separate learning rule.
- **R3 (every aspect self-modified):** The aspiration. 12 prescribed components remain (Section 7.6). Every experiment that prescribed fewer components performed worse (Step 593: removing centering → 0/5). R3 defines the gap between current substrates and the goal.
- **R4 (modifications tested against prior state):** Negative transfer destroys prior capability when untested (Steps 506, 515, 596). Domain isolation (separate edge dicts) is the empirical solution — effectively a per-domain R4 check.
- **R5 (one fixed ground truth):** Theorem 3: without a fixed external ground truth, R3 permits self-modification of evaluation criteria, which is degenerate. The ground truth must be environmental (game death, level transitions).
- **R6 (no deletable parts):** Step 548: 89.5% of Recode splits change argmin action. Every component is behaviorally load-bearing. Redundant components would waste the growth budget (U17).

Alternative frameworks (Schmidhuber's Gödel Machine, open-ended evolution, intrinsic motivation) make different commitments. R1-R6 are not uniquely determined — but each is experimentally motivated, not arbitrary.

3.1 Primitives

The substrate is a triple (f, g, F): - f_s: X → S (state update parameterized by current state s) - g: S → A (action selection) - F: S → (X → S) (meta-rule mapping states to update rules; F IS the frozen frame)

Dynamics: s_{t+1} = F(s_t)(x_t), a_t = g(s_t).

3.2 Structural Rules (R1-R3) and the Core Tension

Prior work: R1 (no external objectives) is the standard unsupervised/self-supervised setting. R2 (adaptation from computation) rules out external optimizers — related to Hebbian learning (Hebb, 1949) where adaptation is local and intrinsic. R3 (self-modification) is formalized by Schmidhuber (2003) as the Gödel machine's self-referential property, though his version requires a proof searcher we do not.

Our formalization:

- **R1:** $f_s(x)$ depends only on s and x. No external signal L enters. The substrate is a closed dynamical system over S given input stream X.
- **R2:** All parameters $\Theta(f) \subseteq S$. The only mechanism modifying Θ is f itself. No gradient ∇L, no external optimizer. The map $t \mapsto s_t$ is generated entirely by iterating f.
- **R3:** $F : S \to (X \to S)$ is non-constant. $\exists s_1 \neq s_2$ such that $F(s_1) \neq F(s_2)$ as functions on X. The update rule depends on the state, not just the data.

Core Tension (Theorem 1): R3 + U7 (dominant amplification) + U22 (convergence kills exploration) produce convergence pressure. U17 (unbounded growth) prevents convergence. Proof: if $s_t \to s^*$, then $\phi(s^*) < \infty$, contradicting U17. Therefore the system has no fixed point. Self-modification is necessary — growth perpetually disrupts the dominant mode.

Relationship to prior work: The no-fixed-point result is a consequence of combining standard dynamical systems theory (spectral convergence) with the growth axiom (U17). The individual pieces are known; the combination producing perpetual non-convergence appears to be our derivation. Closest prior work: Schmidhuber's "asymptotically optimal" self-improvement, which converges — our system provably doesn't.

Tensions: - T1: U7 assumes stationarity that R3 breaks. May need reformulation as "locally amplifies largest-variance component" (instantaneous, not asymptotic). - T2: R3 is ambiguous between "actively modifies operations" and "operations responsive to state." Resolved by the self-modification hierarchy below.

Resolving T2: The Self-Modification Hierarchy **Prior work:** Schmidhuber (1993, "A Self-Referential Weight Matrix") proposed collapsing potentially infinite meta-learning levels into one self-referential system. Kirsch & Schmidhuber (2022, ICML) implement this: a weight matrix that learns to modify its own weights, achieving meta-meta-. . . -learning without explicit hierarchy. The hierarchy of meta-learning levels is well-established: Level 0 (fixed), Level 1 (learning), Level 2 (learning to learn), etc. Our contribution is mapping this hierarchy onto our specific decomposition of F.

Our formalization: Decompose $F(s)(x)$ into three components: - $\pi_s : X \to N$ (encoding — maps observations to nodes) - $u_s : N \times S \to S$ (update — modifies state given a node) - $g_s : S \to A$ (selection — chooses actions)

The **self-modification level** of a substrate is determined by which components' *structure* (not just data) depends on s:

Level	What depends on s	Example	R3?
ℓ_0	Only g_s (action from data)	LSH: fixed hash, argmin over edge counts	Weak only
ℓ_1	g_s and data in u_s	Codebook: entries move via attract, but attract rule fixed	Weak only
ℓ_π	π_s changes structure	Recode: hash routes through learned hyperplanes from transition statistics	**Partial**
ℓ_F	The rule for modifying π itself adapts	No substrate yet. Schmidhuber's SRWM is Level ℓ_F for neural systems.	**Strong**

Key distinction: At ℓ_0 and ℓ_1, the encoding π is fixed at initialization. The system sees through the same lens forever; only what it remembers changes. At ℓ_π, the lens itself changes — an observation that mapped to node n_1 at time t may map to $(n_1, 0)$ at time $t' > t$ because a hyperplane was learned. This is a qualitative jump: the system's perception of the environment changes, not just its memory.

Empirical evidence (Step 542, Recode): ℓ_π produces 5/5 L1 navigation at 5 seeds. But Step 589 (20 seeds, K-controlled) reveals a confound: Recode(K=16) 18/20 = LSH(K=16) 18/20 > LSH(K=12) 13/20. The 5/5 vs 3/3 advantage was K=16 vs K=12, not adaptive splitting vs fixed hashing. Self-modification of π provides a speed advantage at mid-budget ($p<0.05$ at 30-40K checkpoints) but does not improve final success rate at 50K. The refinement algorithm is frozen — the system cannot modify HOW it refines, only WHERE. This is partial R3 in principle, but operationally equivalent to ℓ_0 with the same K.

Relationship to prior work: Schmidhuber's hierarchy (1993) addresses the same question — what level of self-reference does the system have? His solution (self-referential weight matrix) collapses all levels into one. Our framework makes the levels explicit and maps concrete substrates to them. The contribution is taxonomic: categorizing mechanisms by what they modify (g, u, π, F). The predictive claim — that higher levels produce measurably better outcomes — is not supported at 20 seeds (Step 589). The hierarchy describes what is modified, not whether modification helps.

Implication for Theorem 2: Self-observation (Section 4.3) requires extracting new irredundant structure from s. At ℓ_0, the system cannot encode this new structure into π — it can only add edges. At ℓ_π, it can modify π to distinguish states that were previously confused. ℓ_π is necessary but may not be sufficient for full self-observation — the system also needs to modify its update and selection rules (ℓ_F).

3.3 Growth Topology (U3, U17, U20, R6)

Prior work: U3 (zero forgetting) is the catastrophic forgetting constraint from continual learning (McCloskey & Cohen, 1989; French, 1999). U20 (local continuity) is Lipschitz continuity of the mapping — standard in topology-preserving embeddings (Kohonen, 1988; van der Maaten & Hinton, 2008 for t-SNE). R6 (no deletable parts) relates to minimal sufficient statistics (Fisher, 1922).

Our formalization:

- **U3:** $S_t \subseteq S_{t+1}$ (structural inclusion — components added, never removed). Values may change; skeleton only grows.
- **U17:** $\exists \phi : S \to \mathbb{R}_{\geq 0}$ monotonically non-decreasing, with $\lim_{t\to\infty} \phi(s_t) = \infty$.
- **U20:** $\pi : X \to N$ is Lipschitz: $d_N(\pi(x_1), \pi(x_2)) \leq L \cdot d_X(x_1, x_2)$.
- **R6:** For every component $c \in \text{components}(S_t)$, the restricted state $S_t \setminus \{c\}$ fails the ground truth test G.

Propositions: 1. U3 + U17 + R6 $\Rightarrow$ irredundant growth (every new component covers unique territory). 2. U20 + irredundancy $\Rightarrow$ well-separated nodes (Voronoi cells partition X without unnecessary overlap). 3. Growth rate is coupled to observation distribution (environment gates growth). 4. U20 constrains node topology to inherit from X (connected observations $\to$ connected nodes).

Relationship to prior work: The individual constraints are known (catastrophic forgetting, Lipschitz continuity, minimal sufficiency). The combination producing "irredundant growth" — where every new component is both unique and necessary — appears to be our synthesis.

Tension T3: R3 (self-modifying metric) + U20 (continuous mapping) = metric can REFINE (increase resolution) but cannot REARRANGE (swap what's near/far). This constrains the space of allowed self-modifications.

Tension T4: Finite observation space + irredundant growth = node count is bounded. U17 must be satisfied by something other than nodes (edges, in practice). But edge-count growth after node saturation violates R6 (marginal counts are redundant). This leads to Theorem 2 (Section 4.3).

3.4 Action Selection Constraints (U1, U11, U24)

U1: No separate learning and inference modes **Prior work:** Online continual learning (OCL) formalizes exactly this constraint. Aljundi et al. (CVPR 2019, "Task-Free Continual Learning") define systems that learn from a single-pass data stream with no task boundaries. The broader OCL literature (survey: arXiv:2501.04897) emphasizes "real-time adaptation under stringent constraints on data usage." Our U1 is equivalent to the OCL setting.

Our formalization: F is time-invariant. The same meta-rule $F : S \to (X \to S)$ applies at every timestep. There is no mode variable $m \in \{train, infer\}$ that changes F's behavior. Formally: the system's dynamics are a single autonomous dynamical system, not a switched system.

Relationship to prior work: Equivalent to OCL's single-pass constraint. Not novel — properly attributed.

Implications: Combined with R3 (self-modification), U1 says: the system modifies itself using the SAME function it uses to process input. There is no separate training phase where the system is allowed to self-modify more aggressively. This rules out any architecture with explicit "learning rate schedules" or "warmup phases" unless these are derived from the state itself (which would make them R3-compliant).

U24: Exploration and exploitation are opposite operations Prior work: The exploration-exploitation tradeoff is foundational in RL (Sutton & Barto, 2018). The formal impossibility of a single mechanism optimizing both simultaneously is well-established in multi-armed bandit theory (Auer et al., 2002; Lai & Robbins, 1985). Recent work (arXiv:2508.01287, 2025) suggests exploration can emerge from pure exploitation under specific conditions (repeating tasks, long horizons).

Our formalization: Let $g_{explore} : S \to A$ maximize coverage (minimize revisitation) and $g_{exploit} : S \to A$ maximize classification accuracy. Then $g_{explore} \neq g_{exploit}$ in general.

Specifically: $g_{explore} = \text{argmin}_a \sum_n E(c, a, n)$ (least-tried action) and $g_{exploit} = \text{argmax}_a \text{score}(s, a)$ (highest-confidence action). These select opposite actions when the least-explored action is also the least-confident.

Relationship to prior work: This is the standard RL tradeoff. Not novel. Our contribution is empirical confirmation across 720+ experiments that no single g produces both good navigation and good classification (Steps 418, 432, 444b).

U11: Discrimination and navigation require incompatible action selection Prior work: No direct formal precedent found. The RL literature treats navigation and classification as different TASKS but doesn't formalize them as requiring incompatible mechanisms within the same system. The closest work is multi-objective RL, where Pareto-optimal policies for conflicting objectives are studied (Roijers et al., 2013).

Our formalization: Navigation requires $g_{nav}(s) = \text{argmin}_a \text{count}(s, a)$ — the action that maximizes coverage. Classification requires $g_{class}(s) = \text{argmax}_a \text{score}(s, a, \text{label})$ — the action (label assignment) that maximizes match confidence. These are not just "different" — they are NEGATIONS of each other (argmin vs argmax over the same scoring function applied to the same state).

Relationship to prior work: The specific finding — that argmin produces 0% classification (Step 418g) and argmax produces 0% navigation — appears to be our empirical contribution, not previously formalized as a constraint. However, the underlying principle (coverage-maximizing and accuracy-maximizing objectives conflict) is a special case of multi-objective optimization theory.

Implications: A system satisfying R1-R6 must handle BOTH tasks (navigation and classification). Since they require opposite g, the system needs either: (a) a mechanism to SWITCH between g_{nav} and g_{class} based on context — but U1 forbids mode switching; or (b) a single g that somehow serves both objectives simultaneously — but U24 says this is impossible. This creates a genuine tension.

Degree of freedom 11: How does the system resolve the U1 + U11 + U24 tension? One possibility: g depends on s (which it must, by R3), and the STATE determines whether the system's behavior is more exploratory or more exploitative. This is not mode switching (the function g is the same) — it's state-dependent behavior. The system explores when its state indicates uncertainty, and exploits when its state indicates confidence. This is known in the RL literature as Bayesian exploration (Ghavamzadeh et al., 2015).

3.5 Remaining Structural Rules (R4-R6) and Validated Constraints (U7, U16, U22)

R4: Modification tested against prior state **Prior work:** Regression testing in software engineering (Rothermel & Harrold, 1996). In RL, policy improvement is tested against the previous policy (Kakade & Langford, 2002, conservative policy iteration). In self-adaptive systems, runtime testing validates adaptations against pre-adaptation behavior (Fredericks et al., 2018). The formal requirement that a self-modifying system must self-evaluate is implicit in Schmidhuber's Gödel machine (the proof must show the rewrite improves expected utility).

Our formalization: After each modification $s_{t+1} = f_{s_t}(x_t)$, there exists an evaluation $V : S \times S \times X^* \to \{better, worse, same\}$ applied by f itself (not externally) to novel inputs X^*. V is part of F (frozen).

Relationship to prior work: Equivalent to conservative policy iteration's requirement. The novelty, if any, is requiring V to be part of f's dynamics (R2 compliance) rather than an external evaluator. This is the same distinction we make for R1.

R5: One fixed ground truth **Prior work:** In formal verification, the specification is the fixed point against which the system is tested. In Schmidhuber's Gödel machine, the utility function is the fixed external criterion. In evolution, fitness is determined by the environment (fixed, external). The idea that a self-modifying system needs at least one invariant is well-established — without it, the system can trivially "improve" by redefining improvement.

Our formalization: $\exists!\ G : S \to \{0, 1\}$ (ground truth test) such that $G \notin S$ — the system cannot modify G. G is part of F.

Relationship to prior work: Standard. Not novel.

R6: No deletable parts (minimality) **Prior work:** Minimal realizations in control theory (Kalman, 1963) — a state-space representation with no redundant states. Minimal sufficient statistics (Fisher, 1922). Minimal dynamical systems in topological dynamics — a system where every orbit is dense (Scholarpedia). Our R6 is closest to Kalman's minimal realization: no state can be removed without losing input-output behavior.

Our formalization: For every component $c \in \text{components}(S_t)$: $G(S_t \setminus \{c\}) = 0$. Every element is load-bearing.

Relationship to prior work: Equivalent to Kalman's observability + controllability condition in linear systems. For nonlinear growing systems, we are not aware of a standard formalization. The combination with U17 (unbounded growth) is non-standard — Kalman minimality assumes fixed dimension.

U7: Iterated application amplifies dominant features **Prior work:** Power iteration (von Mises, 1929). In linear systems, repeated application of a matrix amplifies the dominant eigenvector. In neural networks, repeated weight application leads to mode collapse (vanishing/exploding gradients, Bengio et al., 1994). In recurrent networks, the issue is well-studied as the echo state property (Jaeger, 2001).

Our formalization: For linearization Df of f around any trajectory point: $\|f^n(s, \cdot) - \lambda_1^n v_1 v_1^T f(s, \cdot)\| \to 0$ as $n \to \infty$, where λ_1, v_1 are the dominant eigenvalue/eigenvector.

Relationship to prior work: This IS power iteration. Known since 1929. Our contribution is observing it empirically across architectures (codebook Step 405, reservoir Steps 438-439) and noting its interaction with R3 and U17 (Theorem 1).

U16: Input must encode differences from expectation **Prior work:** Mean subtraction / centering is standard preprocessing in machine learning (LeCun et al., 1998, "Efficient BackProp"). In neuroscience, predictive coding (Rao & Ballard, 1999) formalizes the idea that neural systems encode prediction errors, not raw stimuli. Whitening (decorrelation + unit variance) extends centering.

Our formalization: The observation x is preprocessed as $x - \mathbb{E}[x]$ before f_s processes it. In our framework, centering is part of F — it's frozen, load-bearing, and confirmed across 2 families (codebook Step 414, LSH Step 453).

Relationship to prior work: Centering is standard (LeCun 1998). The specific finding that it's load-bearing for TWO architecturally distinct families (codebook: prevents cosine saturation; LSH: prevents hash concentration) is our empirical contribution, not the formalization.

Ablation (Step 593): LSH k=12 without centering: 0/5 L1, avg 5 cells (vs 222 cells with centering). The hash collapses to ~5 states without mean subtraction. One line of preprocessing — `x -= x.mean()` — is the difference between a functioning exploration system and a degenerate one. This confirms U16 is not a soft recommendation but a hard requirement.

Tension with R3 (Proposition 7): U16 is frozen — centering is part of the encoding pipeline that R3 requires to be self-modifiable. But removing centering is lethal (0/5 vs 1/5). This creates a necessary constraint on R3: some components are so load-bearing that self-modification of them is functionally suicidal. The system cannot safely explore modifications to centering because every modification kills navigation. R3 must be interpreted as "every component is self-modifiable IN PRINCIPLE" rather than "every component IS modified" — otherwise the system must explore centering ablations, discover they're lethal, and never try again. This is analogous to highly conserved genes: essential for viability, frozen by selection pressure, not by design constraint.

U22: Convergence kills exploration **Prior work:** In bandit theory, a converged policy exploits one arm and never explores others (Lai & Robbins, 1985). In RL, policy convergence to a deterministic policy eliminates exploration (Sutton & Barto, 2018). The formal statement that convergent dynamics in interactive environments produce stationary behavior is well-established.

Our formalization: If $\|s_{t+1} - s_t\| \to 0$, then (a_t) becomes eventually periodic. Mathematically: convergent adaptation makes the environment appear stationary $\to$ trivially predictable $\to$ no learning signal $\to$ no new states.

Relationship to prior work: Known. The specific empirical confirmations (codebook Step 428 score convergence, TemporalPrediction Step 437d weight freezing) are data points for a known phenomenon.

4. Results

4.1 The Core Tension (R3 + U7 + U17 + U22)

Theorem 1: No satisfying system has a fixed point. If $s_t \to s^*$, then $\phi(s^*) < \infty$, contradicting U17. Self-modification is necessary, not optional — growth perpetually disrupts the dominant mode (U7), preventing convergence (U22).

The system oscillates: U7 drives toward dominant mode → U17 growth disrupts the mode via R3 → new dominant mode emerges → repeat. The oscillation period is a degree of freedom.

4.2 Irredundant Growth

U3 + U17 + R6 together require that the system adds new components indefinitely (U17), never removes them (U3), and every component is necessary (R6). Therefore every new component covers unique territory. Combined with U20 (Lipschitz mapping), this means nodes are well-separated — the system builds an increasingly detailed, non-redundant map of the observation space. Growth rate is coupled to the observation distribution: the environment gates growth through the action-observation loop.

4.3 The Self-Observation Requirement

Theorem 2: In a finite environment, U17 + R6 + R1 require self-observation.

Proof: (1) U17 + R6 require infinitely many irredundant components. (2) In a finite environment, irredundant components from external observations are bounded — once every reachable state has a node, further nodes are redundant; edge-count growth is also redundant ($N(c, a, n) = 10^6$ vs $10^6 + 1$ does not change g). R6 is violated. (3) After external information is exhausted, irredundant growth must come from elsewhere. (4) By R1, $f_s(x)$ depends only on s and x. (5) Since x provides no new irredundant information, the only source is s itself. f must extract structure from s not explicitly stored — temporal patterns, graph properties, meta-state. This is self-observation. □

Implication: The current graph + argmin system exits the feasible region after exploration saturates. The Level 2 failure (Section 5.2) is a feasibility violation, not a strategy failure.

Relationship to prior work: Curiosity-driven exploration (Pathak et al., 2017) proposes self-observation as a design choice. We derive it as a mathematical necessity. The R6 mechanism — irredundancy killing degenerate growth — has no analog in the curiosity literature, which adds intrinsic reward (violating R1) rather than requiring every component to be functionally necessary.

4.4 Fixed-Point Conjecture

Conjecture (not proven): The minimal self-observing substrate is a fixed point of F. Applying the system to its own state trajectory reproduces the system's dynamics. This would terminate the potential infinite regress of self-observation (processing state → new state → processing new state → ...). Related to autopoiesis (Maturana & Varela, 1972): the network produces the components that produce the network. Difference: autopoiesis maintains structure; our system grows (U17). Status: open.

Prior work on eigenforms: Kauffman (2023, "Autopoiesis and Eigenform") connects autopoiesis to eigenforms — fixed points of recursive processes where $f(f) = f$. Formal autopoiesis (Letelier et al., 2023) generates self-referential objects with this property. The (M,R)-system (Rosen, 1991) is equivalent: organizational closure where every component is produced by the network of components. Our conjecture is an instance of eigenform theory applied to dynamical substrates.

Concrete mechanism (self-application of F). The substrate alternates between observing the environment and observing itself:

$$s_{t+1} = F(s_t)(x_t) \quad \text{(environment step)}$$
$$s_{t+2} = F(s_{t+1})(\text{enc}(s_{t+1})) \quad \text{(self-observation step)}$$

where $\text{enc} : S \to X$ embeds the state into observation space. For the graph substrate: each cell's edge profile (outgoing counts per action per successor) is a vector that can be hashed through the same LSH. The substrate builds a graph of its own graph — cells with similar exploration patterns share a meta-cell, revealing attractor basins vs frontier regions. No new operations: the same compare-select-store applied to self instead of environment.

The frozen frame cost is one element: the decision to feed $\text{enc}(s)$ back as input. But enc uses the substrate's own encoding pipeline. The self-observation uses the same hash, the same argmin, the same edge accumulation. This is the eigenform: F applied to its own output.

Concrete instantiation. For the graph substrate with $|A| = 4$ actions, define $\text{enc}(c) = [N(c,0), N(c,1), N(c,2), N(c,3)] \in \mathbb{R}^4$, where $N(c,a) = \sum_n E(c,a,n)$ is the total outgoing count for action a from cell c. Hash $\text{enc}(c)$ through 4 random hyperplanes $\to$ 4-bit meta-cell address $\to$ 16 possible meta-cells. Build a meta-graph M with edges accumulated from the self-observation steps. The substrate maintains two parallel graphs: the environment graph G (from hashing game frames) and the meta-graph M (from hashing action count vectors).

Action selection combines both: when G has tied counts (fresh cell, all actions equal), use M's argmin for the meta-cell of the current cell's action profile. This provides **global tie-breaking**: instead of random (Step 615) or always-zero (Step 614), the substrate selects the action that was least tried across ALL cells with similar exploration patterns. This is transfer learning within the graph — cells that look similar in exploration-space share action recommendations.

The meta-graph has 16 nodes $\times$ 4 actions = 64 edges. Negligible overhead. The frozen frame cost: one decision (hash action counts alongside observations). The mechanism is the same compare-select-store applied to $\text{enc}(s)$ instead of x.

Degeneration constraint (Zenil, 2026). Zenil ("On the Limits of Self-Improving in LLMs," arXiv 2601.05280) formalizes recursive self-training as a dynamical system and proves two failure modes: (1) entropy decay — sampling kills distributional diversity (our U22), and (2) variance amplification — without persistent external grounding, distributional drift via random walk (our U7). The prediction: self-improvement degenerates unless the proportion of exogenous signal is maintained asymptotically.

Implication for the eigenform: Pure self-observation $F(s)(\text{enc}(s))$ without environmental input would degenerate per Zenil's theorem. The mechanism MUST interleave environment steps and self-observation steps: $F(s)(x)$ then $F(s)(\text{enc}(s))$. R5's environmental ground truth (game death, level transitions) provides the external grounding that prevents entropy decay. This predicts the

interleaving ratio matters: too much self-observation → degeneration; too little → no benefit. The optimal ratio is a degree of freedom.

Testable predictions (made before experiments, tested in Steps 620-629): 1. Meta-graph tie-breaking vs random tie-breaking: if meta improves L1 success rate, self-observation provides useful signal at the bootstrap stage. **RESULT: No improvement.** Step 620: L1=5/5 matches baseline exactly. Self-observation is inert at L1. 2. Meta-graph bias on saturated cells: if meta selects different actions than per-cell argmin in the attractor basin. **NOT YET TESTED** at the meta-graph level (eigenform tested simpler statistics). 3. If meta-argmin = per-cell argmin everywhere, self-observation via the same F is inert. **CONFIRMED.** Steps 620-626: self-observation via graph statistics produces AVOID/PREFER/NEUTRAL ops (94-99% NEUTRAL). Step 626: freezing ops after 5000 steps doesn't degrade L1. The mechanism produces non-zero output but has no performance effect. 4. Interleaving ratio. **ANSWERED DIFFERENTLY.** Step 621: adaptive M→2000 means self-observation self-terminates — the distribution stabilizes, new observations add no information. The "optimal N" question is moot: the system naturally reduces self-observation frequency to zero.

Proposition 13: Eigenform Inertness Prior work: Kauffman (2009, 2017) shows every transformation in a reflexive domain has a fixed point (eigenform). In our framework, $F(s)(\text{enc}(s))$ is the eigenform — the substrate applying its own transition function to its own state. Kauffman's theory predicts convergence to a stable self-description. Gödel's incompleteness theorem predicts a system cannot prove statements about states outside its own axioms — analogous to our graph containing no information about unvisited nodes.

Our finding (empirical, 10 experiments): Self-observation on the existing graph converges to a fixed point (eigenform) and is inert for navigation. Specifically:

- Step 620: $F(s)(\text{enc}(s))$ computes percentile thresholds on edge-count distributions. AVOID grows from 0% → 8% of nodes. L1=5/5 (matches baseline — no improvement).
- Step 621: Adaptive observation frequency M → 2000. Self-observation self-terminates when the distribution stabilizes. **This is the eigenform fixed point:** $F(s)(\text{enc}(s)) \approx s$ — self-observation produces no new information.
- Step 626: Freezing ops after 5000 steps has zero effect on L1. The eigenform is inert even when active.
- Step 629: L2=0/5 even with L1-success tagging. Self-observation of the EXISTING graph cannot produce information about UNVISITED states.
- Step 625: Chain benchmark P3 is 7-53x SLOWER with eigenform. AVOID contaminates known paths — the eigenform HURTS when it acts, because it avoids familiar territory without knowing what unfamiliar territory contains.

Formalization: Let $G = (N, E, \text{count})$ be the graph state. Self-observation $h : S \to S$ computes statistics $\sigma(G)$ that are functions of $\{(n, a, \text{count}(n, a)) : n \in N, a \in A\}$. Level 2 requires reaching nodes $n' \notin N$. For any statistic σ computable from G, σ is a function of visited states only. No function of $\{n \in N\}$ produces information about $\{n' \notin N\}$ without a model that predicts observations beyond the frontier.

Proposition 13 (Eigenform Inertness): Self-observation via $F(s)(\text{enc}(s))$ is necessary (Theorem 2) but not sufficient for Level 2+. The gap is informational: the graph stores the past; Level 2

requires predicting the future. Introspection identifies structure in visited states but cannot navigate to unvisited states. This is the same wall as the noisy TV barrier (Section 4.5) from a different angle — both show that signals derived from the existing state cannot guide exploration beyond it.

Relationship to prior work: Kauffman's eigenform theory predicts convergence — confirmed (Step 621: M→2000). Gödel's incompleteness predicts self-referential limitations — confirmed (Step 629: L2=0/5). Our contribution: the empirical demonstration that eigenform inertness is not an implementation failure but a structural limitation. Self-observation MUST be supplemented by a mechanism that predicts unvisited states — a world model or forward prediction.

Implications for R3: R3 requires every aspect to be self-modified. Eigenform provides one form of self-modification (AVOID/PREFER ops derived from state). But eigenform alone cannot achieve L2. R3 compliance at L2+ requires something beyond self-observation: state-derived prediction of UNSEEN observations. This is the I1 gap (representation discovery) expressed in the R3 framework.

Degrees of freedom: What predicts unvisited states? Two candidates: (1) frame-delta prediction (Steps 630-631, killed at k=16 — too fine-grained); (2) forward model from transition statistics (if action A from node N often leads to observation class C, predict C for untried actions from similar nodes). Neither tested successfully.

4.5 Argmin Robustness and the Noisy TV Barrier

Prior work: The noisy TV problem (Burda et al., 2019, ICLR) identifies a failure mode of curiosity-driven exploration: agents rewarded for prediction error or novelty are attracted to irreducibly stochastic transitions (a "noisy TV") because these transitions can never be predicted accurately. Random Network Distillation (RND, Burda et al., 2018) addresses this by using a fixed random network as a prediction target, making the bonus deterministic. Count-based exploration (Bellemare et al., 2016) is known to be robust to noisy TV because visit counts don't distinguish high-variance from low-variance transitions.

Our finding (empirical, not a theorem): In R1-compliant systems (no external reward signal), the noisy TV problem is universal across ALL targeted exploration strategies, not just curiosity. We tested 6 independent strategies against pure argmin on LS20:

Strategy	Mechanism	Result	Failure mode
Argmin (baseline)	$g(s) = \text{argmin}_a N(s,a)$	10/10	—
Destination novelty	$g(s) = \text{argmax}_a \text{novelty}(s'\|s,a)$	1/10	Attracted to rare death states
Prediction error	$g(s) = \text{argmax}_a \|s' - \hat{s}'\|$	0/10	Death is maximally unpredictable
Softmax temperature	$P(a) \propto \exp(-N(s,a)/T)$	2/3	Stochasticity without benefit
Entropy-seeking	$g(s) = \text{argmax}_a H(s'\|s,a)$	0/3	Noisy TV: death = max entropy
UCB1	$g(s) = \text{argmax}_a(\bar{r}_a + c\sqrt{\ln t/N_a})$	2/3	Degenerates to argmin (no reward)
Global novelty	$g(s) = \text{argmax}_a(1/N_{\text{global}}(s'))$	6/10	Same count, different seeds

Steps 477-482, 539-541. 6 strategies tested, all worse than or equal to argmin.

Why argmin is robust: Define a transition as *reducible* if $H(s'|s,a) \to 0$ with sufficient observations, and *irreducible* if $H(s'|s,a) > \epsilon$ for all sample sizes (e.g., death transitions, stochastic resets). Any strategy g that selects actions based on model uncertainty (prediction error, entropy, novelty) preferentially selects actions leading to irreducible transitions, because these maximize the quality signal. Argmin $g(s) = \text{argmin}_a N(s,a)$ is immune: visit counts accumulate equally on reducible and irreducible transitions. The cost is slower exploration of structured regions; the benefit is avoiding traps.

Formalization: Let $\mathcal{A}_{\text{irr}}(s) = \{a : H(s'|s,a) > \epsilon, \forall \text{ sample sizes}\}$ be the set of actions with irreducible stochasticity from state s. For any strategy g that maximizes a signal $q(s,a)$ where q is monotone in model uncertainty: $\Pr[g(s) \in \mathcal{A}_{\text{irr}}(s)] \geq |\mathcal{A}_{\text{irr}}(s)|/|\mathcal{A}|$. For argmin: $\Pr[g_{\text{argmin}}(s) \in \mathcal{A}_{\text{irr}}(s)] = |\mathcal{A}_{\text{irr}}(s)|/|\mathcal{A}|$ (uniform in the limit). Targeted strategies have strictly higher probability of selecting irreducible transitions; argmin treats them equally.

Relationship to prior work: The noisy TV problem is known (Burda et al., 2019). Count-based robustness is known (Bellemare et al., 2016). Our contribution is empirical: (1) confirming the noisy TV problem applies to R1-compliant systems where death transitions are the "noisy TV," and (2) showing that 6 independent attempts to improve on argmin all fail for the same reason. The claim "argmin is locally optimal among memoryless count-based strategies in environments with irreducible stochasticity" is our empirical finding, not a published theorem.

Implication for R3: Self-observation (Theorem 2) requires the system to extract new structure from its own state after exploration saturates. But any mechanism that TARGETS specific state structures for observation will be vulnerable to the noisy TV problem — self-referential noise (patterns that look complex but carry no usable signal). The self-observation mechanism must be as blind as argmin is to transition entropy.

Proposition 15: Perception-Action Decoupling **Prior work:** In POMDPs, the observation function determines which hidden states the agent can distinguish. State aliasing (Whitehead & Ballard, 1991) occurs when the observation mapping conflates distinct hidden states. Computing optimal memoryless POMDP policies is NP-hard (Littman, 1994). In POMDPs with state aliasing, stochastic policies can be strictly better than deterministic ones (Singh et al., 1994). Count-based exploration (Bellemare et al., 2016; Tang et al., 2017) is robust to observation noise but sensitive to hash granularity — "important aspects of a good hash function include having appropriate granularity" (Tang et al., 2017).

Our finding (empirical): LS20 Level 1 is a hidden-state conjunction problem: the game has hidden state variables inaccessible to the substrate (Step 654). L1 triggers when the agent is at the exit cell AND the hidden state satisfies a conjunction condition. The agent visits the exit cell avg 152 times before L1 triggers (Step 652). L1 action sequences are near-uniform (entropy $H = 1.948/2.0$, Step 657). Different seeds unlock different conjunctions under different action orderings (argmin-only: 3/20, random-only: 3/20, both: 1/20, Step 653).

Extended argmin optimality (Steps 652-671, 20+ experiments). Beyond the 6 strategies in Section 4.5, we tested 14 additional interventions targeting the POMDP structure:

Intervention class	Steps	Result
Edge-state enrichment (4 variants)	640-648	All KILL or MARGINAL. Profile uniformity under argmin destroys transfer signal.
Path conditioning (2nd-order Markov)	649	MARGINAL. 1.09x speedup = noise.
Outcome-conditioned selection	667	KILL 0/10. Key explosion — too many (cell, outcome) tuples.
Visit-count-dependent behavior	668	MARGINAL 5/10. Lost 3 baseline seeds. Random break too coarse.
Gaussian variance refinement	669	MARGINAL 5/10. 145x on successes; too many hi-var cells on failures.
Alternating argmin/random	670	5/20. Found 3 NEW seeds but lost 5 union seeds. Budget-limited.
1-step world model (splatter)	671	KILL 0/10. Noisy TV: death = max frame diff. Same wall as 477-482.

Pattern: Every intervention helps some seeds and loses others. No perturbation of g consistently improves L1 rate. The interventions that help fastest seeds (669: s3 at 285x, s4 at 6.4x) help the seeds where hash resolution ALREADY separates hidden states at the exit cell (Step 664: exit entropy ~2.0 for fast seeds). The interventions cannot help seeds where the hash conflates all hidden states (exit entropy = 0.0).

Formalization: Let $\pi : X \to N$ be the observation mapping (LSH hash), $h \in H$ the hidden state, and $T \subset N \times H$ the L1 target set. Define: - $R(\pi, g)$ = L1 rate under mapping π and action selection g - $k_n = |\{h : (n, h) \in \text{reachable}\}|$ = number of hidden states mapped to node n - $n^* = \text{proj}_N(T)$ = exit node

Proposition 15 (Perception-Action Decoupling): For hidden-state conjunction problems, $R(\pi, g)$ is primarily determined by π, not g. Specifically:

1. For any g satisfying asymptotic uniform coverage of N (including argmin, random, and their perturbations): $R(\pi, g) \approx R(\pi)$. The L1 rate depends on whether π resolves hidden states at the exit cell, not on the action selection strategy.

2. The per-visit L1 probability at the exit cell is $\approx 1/k_{n^*}$ — the reciprocal of the number of hidden states aliased at the exit node. Fast seeds: k_{n^*} is small (hash resolves, L1 in <1000 steps). Slow seeds: k_{n^*} is large (hash conflates, L1 at 22K-24K steps). (Steps 664-665, correlation — causal confirmation pending.)

3. No perturbation of g can reduce k_{n^*}. Only modifications to π (finer hash, self-refinement, centering) can improve resolution at critical cells.

Relationship to prior work: The NP-hardness of optimal memoryless POMDP policies (Littman, 1994) explains WHY no tested intervention consistently beats argmin — there is no efficiently computable improvement. Our contribution is empirical: confirming this prediction across 20+ interventions in a specific interactive game, and identifying the observation mapping resolution at critical cells as the measurable bottleneck.

Consistency with Theorem 2: Theorem 2 requires self-observation after exploration saturates. Proposition 15 shows the self-observation must target π (the observation mapping), not g (action selection). Recode (Step 542, Section 5.4.5) achieves exactly this — ℓ_π modification of π from transition statistics. This is the only tested mechanism that consistently improves L1 reliability (5/5 vs 3/3). The convergence: Theorem 2 says self-observation is necessary → Proposition 15 says it must target π → Recode demonstrates ℓ_π is achievable.

Degrees of freedom — RESOLVED by Proposition 16: Transition-inconsistency refinement (Step 674) achieves targeted π-refinement at critical cells without foreknowledge. Cells with inconsistent transitions ($I(n) \geq 2$) are refined with finer hash. The exit cell has the highest inconsistency because hidden-state conjunctions create transition variability there. Result: 17/20 L1 at 25s (Step 690, 20 seeds). See Proposition 16 for formalization.

Corollary: L1/L2 Asymmetry Finding (Steps 682-686, empirical): Proposition 15's perception-action decoupling holds for L1 but NOT for L2. The qualitative difference:

- **L1 aliasing is bounded.** The number of aliased cells (cells with inconsistent transitions) plateaus at 83-313 depending on seed. The fine graph (k=20 at aliased cells) accumulates useful counts within the 25s budget. Step 674: 9/10 L1.
- **L2 aliasing is unbounded.** After L1, aliased cells grow monotonically: 2→439 (seed 8, 572K steps), 48→445 (seed 3, 535K steps). The fine graph becomes sparser as more cells compete for budget. The mechanism adapts (detects new aliased cells post-L1) but the adaptation diverges — count accumulation can never keep pace with the growing aliased set.

Formalization: Let $A(t) = |\{n \in N : |\text{succ}(n, a)| \geq 2 \text{ for some } a\}|$ be the aliased cell count at step t. For L1: $A(t) \to A^*$ (bounded, seed-dependent). For L2: $A(t) \sim c \cdot t^\alpha$ for $\alpha > 0$ (unbounded growth). The per-aliased-cell visit rate is $\sim 1/A(t)$. Count-based action selection requires $\Omega(\log |\mathcal{A}|)$ visits per cell to distinguish actions. When $A(t) \gg A^*_{L1}$, the per-cell visit rate drops below the threshold for useful count accumulation.

Implication: L2 requires a mechanism that can navigate toward a goal region DESPITE unresolved perceptual aliasing — not one that resolves all aliasing before navigating. This is the L2 energy mechanic (iri sprites, Steps 556-557): the agent must navigate TO energy sources before dying, then proceed to the exit. Exhaustive coverage via argmin (which works for L1 because L1's

exit is findable by random coverage) fails for L2 because the energy constraint imposes a TIME LIMIT per episode.

Cross-game confirmation (Step 691). FT09 with 674 at 300s: L1=5/5, L2=0/5. Aliased cells are FROZEN post-L1 (1-4 cells, unchanged over 1.9M steps). Compare LS20 (Step 686): aliased grows 2→471. The L2 wall exists on BOTH games despite fundamentally different aliasing dynamics — FT09 has near-zero aliasing yet L2 is equally unreachable. This proves L2 is not about disambiguation capacity. The substrate is already fully disambiguated on FT09 and still cannot reach L2.

Connection to Proposition 13 (Eigenform Inertness): Self-observation identifies structure in visited states but cannot guide navigation to unvisited states. For L2, the unvisited states include energy sources. The mechanism must predict WHERE energy sources are from partial observations — a world model, not a coverage strategy. This sharpens the L2 problem: the bottleneck is not perception (π) and not action selection (g), but prediction of unvisited states from partial observations. This is the transition from ℓ_π (perception refinement) to ℓ_F (rule modification) — the system must modify its UPDATE RULE to incorporate predictions, not just refine its mapping.

Connection to Section 2.11 (Renormalization Group): The L2 wall as a phase transition is confirmed by Step 691: the L1 effective theory (argmin + transition-inconsistency refinement) has the SAME structure on both games but neither reaches L2. L2 requires different relevant operators — not finer resolution of the same operators. This is precisely an RG phase boundary: the universality class changes at L2.

Proposition 16: Transition-Inconsistency Refinement (Q20 Resolved) Prior work: Perceptual aliasing — when the observation mapping conflates distinct hidden states — was identified by Whitehead & Ballard (1991). Belief-state POMDPs maintain probability distributions over hidden states (Kaelbling et al., 1998). McCallum's utile distinction (1996) splits observation histories when different histories lead to different values. These approaches require either explicit belief-state computation (intractable for large state spaces) or reward-based splitting (violates R1).

Our finding (Steps 672-690, empirical): Transition inconsistency — when a cell has multiple distinct successors under the same partition — is a sufficient, R1-compliant signal for targeted π-refinement. Define observed inconsistency at cell n after τ visits:

$$I_\tau(n) = |\{n' \in N : \exists t \leq \tau, \pi(x_t) = n \wedge \pi(x_{t+1}) = n'\}|$$

Proposition 16: For a POMDP with deterministic hidden-state transitions $T_h : S_h \times A \to S_h$, if $I_\tau(n) \geq 2$ with sufficient visits ($\tau \geq \tau_{\min}$), then cell n conflates ≥ 2 distinct hidden states.

Proof: If $\pi^{-1}(n) \cap \text{image}(O)$ contains a single observation x^*, then all visits to n see x^*, which maps to a single hidden state s^*. Under deterministic T_h, each action a produces a unique successor $s' = T_h(s^*, a)$, mapped to a unique cell $n' = \pi(O(s'))$. At most $|A|$ successors, but each action always produces the SAME successor. Thus $I(n)$ is bounded by the number of distinct $\pi(O(T_h(s^*, a)))$ across actions, which is deterministic. If $I_\tau(n) \geq 2$ for the SAME action a (same action yielding different successors), then n must contain ≥ 2 hidden states. QED.

Mechanism (Step 674): Mark cells with $I(n) \geq 2$ and visits$(n) \geq 3$ as "aliased." Apply fine hash (k=20) only at aliased cells; use coarse hash (k=12) elsewhere. Result: **17/20 L1 at 25s** (Step 690, definitive 20-seed sweep). Cross-game: FT09 5/5 (Step 680). The binary criterion (any inconsistency) outperforms ranked or capped variants (Steps 674b/c/d: all 8/10).

Why it works without foreknowledge: The exit cell has the highest $I(n)$ because it is where the hidden-state conjunction creates the most transition variability. The mechanism doesn't know the exit cell is task-critical; it discovers this from transition statistics. Selective refinement at high-I cells concentrates resolution where aliasing is worst — which correlates with task-critical locations. This resolves Proposition 15's open question: R1-compliant targeted π-refinement IS achievable via transition inconsistency, without environmental feedback or foreknowledge.

Relationship to prior work: The inconsistency signal is a special case of belief-state refinement: non-deterministic transitions at a cell imply multiple hidden states, which is precisely what belief states track. Our contribution is the APPLICATION: using this as an online, R1-compliant partition refinement mechanism in a self-modifying substrate. No prior work we found combines (1) transition-based aliasing detection, (2) selective partition refinement at runtime, and (3) no external objective in a single online system.

Limitations: The mechanism works for L1 (bounded aliasing, 83-313 cells) but diverges for L2 (unbounded aliasing, 439+ cells — see L1/L2 Asymmetry corollary). Transition inconsistency detects aliased states but cannot RESOLVE the aliasing when the aliased set grows faster than counts accumulate.

4.6 Constructive Characterization of the Feasible Region

Combining all formalized constraints, we characterize the class of F that satisfies R1-R6 + validated U-constraints simultaneously.

The minimal frozen frame F decomposes as:

$$F(s)(x) = \texttt{store}(s, \texttt{select}(s, \texttt{compare}(s, x)))$$

where: - $\texttt{compare}(s, x)$: produces similarity/distance scores between x and elements of s - $\texttt{select}(s, \text{scores})$: chooses an element of s (or triggers creation of a new element) - $\texttt{store}(s, \text{selection})$: updates s with the result

Constraints on each operation:

Operation	Must satisfy	Cannot do
compare	Lipschitz in x (U20). Varies with s (R3). Encodes $x - \mathbb{E}[x]$ (U16).	Rearrange topology (T3). Use external signal (R1).
select	Deterministic given s and scores. Hard selection, not soft blend (U8, provisional).	Converge to fixed policy (U22 + Thm 1).
store	Only add, never remove (U3). Irredundant (R6). Unbounded (U17).	Create redundant components. Delete existing components.

Phase structure (from Theorem 1):

The trajectory of s has two alternating phases: - **Phase A (amplification):** U7 drives toward the dominant mode. compare and select become increasingly predictable. The system exploits

accumulated structure. - **Phase B (disruption):** U17 growth adds new components. Via R3, new components change compare and select. The dominant mode shifts. The system explores new structure.

The oscillation between phases is perpetual (Theorem 1). The PERIOD is a degree of freedom — fast oscillation produces chaotic behavior, slow oscillation produces near-convergent behavior with occasional disruptions.

After exploration saturates (Theorem 2):

Phase B must draw new components from s itself (self-observation), not from x. The store operation must be able to create components of TYPE different from the original observation-derived components — meta-components that represent patterns, compressions, or properties of the existing state.

This means S is not homogeneous. It contains at least two kinds of elements: 1. **Observation-derived** components (nodes from $\pi(x)$, edges from transitions) — bounded by the finite environment. 2. **State-derived** components (patterns extracted from the trajectory of type-1 components) — potentially unbounded, satisfying U17 after saturation.

What remains undetermined:

The decomposition into compare-select-store is the STRUCTURE of F. The specific IMPLEMENTATIONS of each operation are the degrees of freedom (Section 6). The feasible region is the set of all $(compare, select, store)$ triples that satisfy the constraints in the table above.

Non-emptiness: The feasible region for compare-select-store with observation-derived components is non-empty — the graph + k-means + argmin system occupies it (Section 5.1). The feasible region INCLUDING state-derived components (self-observation) is the open question. No system has been shown to satisfy R6 after exploration saturates.

Dual-signal conjecture (Step 581, pending): A substrate with per-element default operations and per-element prediction error signals may satisfy R3 if: (1) predictions are stored in element state, not external code; (2) error signals modify predictions, not the default operation; (3) modified predictions change which element fires next. The frozen frame reduces to three operations: similarity, error computation, and default selection. This is R6 at the operation level — the letter's conjecture that irreducibility applies to operations, not data. The data (predictions) self-modify through interaction. The operations (similarity, error, default) are the irreducible interpreter. Inspired by cerebellar dual-signal architecture (climbing fiber error + Purkinje cell default).

Relationship to prior work: The compare-select-store decomposition resembles the read-match-write cycle of Neural Turing Machines (Graves et al., 2014) and the content-based addressing of memory-augmented neural networks. The key difference: in NTMs, the controller is trained by backpropagation (violates R1/R2). In our framework, the controller IS F — frozen, minimal, and non-trainable. The state s does all the adapting.

4.7 Interpreter Entailment (Proposition 14)

The question (posed 2026-03-21): Section 4.6 identifies the minimal frozen frame as the interpreter: compare-select-store. When is this interpreter *entailed* by the system — a logical consequence of R1-R6 + task structure — rather than a designer choice?

Prior work: Rosen's (M,R)-systems (1991) formalize organizational closure as a cycle of efficient causation: metabolism $f : A \to B$, repair $\Phi_f : B \to H(A, B)$, and replication $\beta : B \to H(B, H(A, B))$, forming the entailment cycle $f \to \beta \to \Phi_f \to f$. Critically, Letelier et al. (2006) and Cárdenas et al. (2010) showed that **the existence of β does not follow from the architecture of (M,R)-systems** — it must be postulated. The closure cycle is closed by assumption, not by derivation. Independently, Bauer (2016) proved that self-interpreters do not exist in System T (a total typed lambda calculus): a self-interpreter implies fixed-point operators at all types, which total languages cannot have. Self-interpretation requires general recursion. Recent work (Letelier et al., 2023, BioSystems) gives exact solutions for functional closure equations (FCEs) in restricted cases, partially addressing Rosen's contested conjecture that closure to efficient causation has no computable models.

Our formalization: We distinguish two senses of "entailment":

1. **Abstract entailment:** The OPERATIONS {compare, select, store, modify} are logically necessary given R1-R6 + U3:

   - R1 (no external objectives) → the system must distinguish inputs autonomously → **compare**
   - R2 (adaptation from computation) → discriminative action choice IS adaptation → **select**
   - U3 (zero forgetting) → persistent accumulation required → **store**
   - R3 (self-modification) → stored procedures must change → **modify**

   Each operation is entailed by a different rule. Removing any operation violates the corresponding rule. The abstract interpreter is the intersection of all rules.

2. **Concrete entailment:** The IMPLEMENTATIONS (cosine similarity, argmin, entry creation, attract-update) are NOT entailed. Multiple implementations satisfy the abstract requirements (confirmed: 4 families navigate with different implementations of compare, Steps 521-525). The specific choice is the frozen frame.

Proposition 14 (Interpreter Entailment): The abstract interpreter {compare, select, store, modify} is entailed by R1-R6 + U3. The concrete interpreter (specific implementations) is not. The gap between abstract and concrete entailment is exactly the R3 problem: can the system discover its own implementations?

Connection to Rosen: Rosen's β-map asks whether the system produces its own components at runtime (constructive closure). Our abstract entailment asks whether the components are logically necessary (deductive closure). These are different questions. Logical necessity does not imply constructive production. A system can require an interpreter without being able to build one. The parallel is precise: Letelier et al. (2006) showed that for Rosen's closure to have a solution, the set of admissible metabolism functions must be "drastically smaller" than Rosen assumed. Our Theorem 1 (no fixed points) is the analogous result — not every state-update rule is compatible with R1-R6. Both frameworks converge: self-referential closure constrains the space of valid systems far more than naive analysis suggests.

Connection to computability: Rosen conjectured that closure to efficient causation has no computable models. Mossio & Longo (2009) resolved this: closure IS computable. They exhibited an expression of closure to efficient causation in lambda calculus, using reflexive objects in Cartesian

closed categories (Scott's D_∞ models). The construction is fully effective — self-referential closure requires reflexive domains (spaces where functions are elements), not non-computability. Our system is computable and operates in a reflexive domain: the state space S contains edge counts that the interpreter reads as instructions. The Rosen conjecture, as originally stated, is false. The real barrier is not computability but EXPRESSIVENESS — whether the reflexive domain is rich enough to encode the operations needed (Q21).

Connection to Bauer and Brown-Palsberg: If our system's dynamics were total (always halting), the concrete interpreter could not be self-produced (Bauer's result for System T). Our system is non-total (s_t evolves indefinitely), so the Bauer obstruction doesn't apply directly. Brown & Palsberg (2016) showed that self-representation IS achievable in sufficiently powerful total type systems (System U), demonstrating the obstruction is about type-system expressiveness, not totality per se. For our framework: the question is not whether self-interpretation is possible in principle (it is, in reflexive domains) but whether our SPECIFIC interpreter + state space combination has sufficient expressiveness.

Connection to constraint closure: Montévil & Mossio (2015) formalize biological organization as "closure of constraints" — a regime where each constraint maintains the conditions for other constraints' existence. Our constraint map (U1-U26, I1-I9) is structurally analogous: each validated constraint narrows the feasible region, and the constraints collectively define what a satisfying system must be. The key difference: our constraints are EXTERNALLY imposed (R1-R6 by the team, U-constraints by experimental evidence). Montévil-Mossio closure is INTERNALLY maintained (the system produces its own constraints). A system achieving R3 would need to transition from external to internal constraint maintenance — discovering not just implementations but the REASONS those implementations are required.

The synthesis: Abstract entailment + R3 creates a constructive demand: the system must discover concrete implementations of operations that are abstractly necessary. This is not a search for WHAT to do (the abstract operations are determined) but HOW to do it (the implementations are free). The frozen frame is the HOW; R3 asks it to become adaptive.

Implications: 1. ℓ_F (rule-modification) requires $S \supseteq \mathrm{repr}(F)$: the state space must contain representations of the meta-rule. Without this representational capacity, the system cannot modify what it cannot represent. 2. The codebook's state ($\mathbb{R}^d$ centroids) cannot represent update rules $\rightarrow \ell_F$ is structurally impossible for the codebook. Not an engineering problem — a type error. 3. A program-bearing state (op-codes as edge data, Steps 581d-582) CAN represent operations $\rightarrow \ell_F$ becomes possible in principle. 4. Whether the constructive gap is bridgeable — whether a system can discover its own implementations of compare-select-store from interaction alone — is the central open question. Rosen says no (for biological systems, contested). Bauer says no (for total systems, proved). Our evidence (720+ experiments, 0 substrates at ℓ_F) is consistent with both pessimistic answers but has not established impossibility.

Corollary (Dissolution of the R3-Navigation Coupling): Proposition 11 found that the frozen frame and navigation capability are "structurally coupled" — the 5 unjustified elements that enable navigation ARE the elements R3 requires the system to self-modify. Interpreter entailment dissolves this tension. The abstract operations {compare, select, store} are BOTH the frozen frame AND the navigation mechanism. They are coupled because they are the SAME thing at the abstract level. R3 does not require modifying these abstract operations (which would destroy navigation). R3 requires discovering IMPLEMENTATIONS of these operations — concrete comparison functions, selection policies, storage formats — that are distinct from the abstract structure. The Proposition 11 coupling is between abstract structure (fixed, entailed) and concrete implementation

(variable, discoverable). This is not a conflict to be resolved but a design constraint: the system must search within the space of implementations, not within the space of operations.

Degrees of freedom: The representational capacity of S — what operations can be encoded as state that the interpreter reads and executes — determines whether interpreter entailment is achievable (DoF 9).

5. Experimental Evidence

5.1 Navigation (720+ experiments)

All 3 ARC-AGI-3 games solved at Level 1:

Game	Mechanism	Result	Steps
LS20 Level 1	LSH k=12 + centered_enc + avgpool16, 4 actions, argmin	Plain: 11/20 at 25s, 16/20 at 120K. **674+running-mean: 20/20 at 25s.** 674 frame-local: 17/20 at 25s, 20/20 at 120K.	126 (easy) to 27K (hard)
FT09 Level 1	LSH k=12 + centered_enc + avgpool16, 6 actions, argmin	Plain: 8/20 at 25s = 8/20 at 120K (binary). 674 frame-local: 17/20. **674+running-mean: 20/20.**	566-12603
FT09 Level 1	k-means n=300, 69 actions (64 click grid + 5 simple), argmin	3/3 at 50K	157, 2035, 3840
VC33	k-means n=50, 3 actions (zone discovery), argmin	3/3 at 30K	18, 82, 144

Unifying mechanism: graph + edge-count argmin + correct action decomposition. The mapping (observation → node) and action space decomposition are the variables. The graph + argmin is the constant across all winning configurations.

Action space finding: The native API presents 6 actions (FT09) and 1 action (VC33). The effective action space is 69 and 3 respectively. The mismatch between API-visible actions and effective actions was the key unsolved problem from Steps 467-502. Zone discovery (Step 505) and grid-click expansion (Step 503) resolve it.

Apparent ceiling resolved: The 6/10 ceiling at 50K on LS20 (Steps 459-483) was a step budget artifact. Hard seeds need 35K-115K steps vs 5-20K for easy seeds. Step 484 confirmed 10/10 at 200K. The 6/10 figure should not be cited as a fundamental limit.

POMDP reframing and selective -refinement (Steps 652-690). L1 is a hidden-state conjunction problem: the agent visits the exit cell avg 152 times before L1 triggers (Step 652). The

bottleneck is hash resolution at the exit cell, not action selection (Proposition 15). Step 672 (diagnostic): k=20 separates hidden states that k=12 conflates — slow seeds have 20-43 k=20 sub-cells at the exit cell vs 6.79 average. **Step 674 (transition-triggered dual-hash): 17/20 L1 at 25s (Step 690), 20/20 at 120K (Step 692 — complete L1 coverage, +11 seeds vs baseline 9/10).** Cells with inconsistent transitions (|successor_set| 2) get fine k=20 hash; all others use k=12. Seed 8: 192x faster (24235→126). The aliased-cell count determines success: <130 aliased cells → faster, >180 → slower or fails. 3 missing seeds (7, 12, 16) have 181-214 aliased cells — likely budget-limited, not mechanism failures. This is _: the observation mapping is refined from the system's own transition statistics, R1-compliantly. Cross-game: FT09 5/5 (Step 680, aliased=1-4), VC33 0/5 (Step 681 — irrelevant, VC33 bottleneck is action decomposition not aliasing).

Encoding requirements: centered_enc ($x-\text{mean}(x)$) is load-bearing across 2 families (codebook: prevents cosine saturation; LSH: prevents hash concentration). avgpool16 (64×64 → 16×16 = 256D) is required for LS20 — raw 4096D with k=16 gives 1/5 vs 6/10 for avgpool16 (Step 466).

5.2 Level 2 Failure as Feasibility Violation

LS20 Level 2 is the only unsolved problem. The "259-cell ceiling" (Steps 486-492) was a TIME LIMIT, not a topological barrier. Steps 528-529 showed sublinear growth continues: 259 cells at 50K → 439 at 740K, growth rate ~2 cells/100K at 740K. At k=16, reachable set expands to 1094 cells at 200K (Step 531), 1267 with self-refinement (Step 542). Level 2 remains unreached at all tested budgets and granularities.

Two families confirm identical behavior: - LSH k=12: 439 cells at 740K, 0/N Level 2 (Steps 486-492, 528-529) - L2 k-means n=300: 286-cell plateau, 0/3 Level 2 (Step 493) - Recode k=16: 1267 cells at 500K, 0/5 Level 2 (Step 542)

All edge manipulations reduce coverage below the pure argmin baseline (argmin=259 > decay=241 > death-avoidance=227 > death-seeking=196, Steps 489-492). Six targeted exploration strategies all perform worse than argmin (Section 4.5). The reward region is beyond the argmin-reachable frontier regardless of mapping architecture, partition granularity, or budget.

The state graph reveals why (Step 550). At 500K steps with Recode k=16: 942 live cells, fully connected (1 component), 13695 edges. The agent circulates in a 364-node **active set** (29% of nodes). 834 nodes (67%) were visited early and abandoned. 134 (node, action) pairs have low entropy ($H < 0.1$) and only 10 observations each — these are the **deterministic frontier**: edges the agent has barely tried, leading to potentially unexplored regions. 67% of all (node, action) pairs have $H > 1.0$ (noisy TV transitions). Argmin is locally optimal (picks the least-visited action from the current node) but globally trapped: from any node in the active set, all actions are well-explored (> 100 observations), so argmin cycles within the attractor. The abandoned nodes with deterministic frontier edges are unreachable because argmin never drives the agent toward them.

Implication: L2 is not a resolution problem (942 cells is more than enough). It is not a budget problem (growth rate is ~2 cells/100K at 740K). It is a **policy problem**: argmin creates an attractor basin in the active set, and the L2 path requires escaping this basin to reach the deterministic frontier in the abandoned set. This directly validates Theorem 2: self-observation is needed to detect the attractor and escape it. ℓ_π (partition refinement) refines within the attractor (Step 549) but cannot break out.

Growth law. Fitting $R(t) = C \cdot t^\alpha$ to the LSH k=12 data (259 cells at 50K, 439 at 740K) gives $\alpha \approx 0.20$. At k=16: 1094 at 200K, 1149 at 500K gives $\alpha \approx 0.05$. The exponent reflects the spectral

gap of the explored graph under argmin dynamics — a small spectral gap (deep attractor basin) produces small α. The 364-node active set (39% of the explored graph) acts as a low-conductance trap: the agent's return time to the frontier grows faster than the frontier itself expands. PAC-MDP theory (Strehl & Littman, 2008) guarantees all states are visited in $O(N^2A)$ steps, but the effective α determines whether this is practically reachable within budget.

Relationship to Section 4: Edge counts grow (U17 formally satisfied) but marginal counts in the active set are redundant (R6 violated). The 134 deterministic frontier edges are the non-redundant growth targets — but argmin cannot reach them.

5.3 Architecture Family Summary

12 families tested across 720+ experiments.

Family	Experiments	Navigation result	Kill reason
Codebook / LVQ	~435	LS20 Level 1 won (Step 82). FT09 won (~Step 83, 69 click classes). VC33 won (Step 375, 3 zones). Level 2 never solved.	Reached capability ceiling. Codebook ban 2026-03-18 (directive). S-class constraints (cosine-specific). Note: Step 375 VC33 win used handcrafted timer encoding (row 0 pixel count) — game-specific, not general. General zone mechanism confirmed by Step 505.
Reservoir	~20	0/N. Steps 438-465.	Rank-1 collapse (U7). Temporal inconsistency = local continuity failure (U20). Cells don't map nearby observations to the same node.
Grid graph	8	0/N. Steps 446-451.	No local continuity: grid topology is arbitrary, not observation-derived. Direct U20 violation.
LSH	~30	LS20: 10/10 at 200K (Step 484). FT09/VC33: topology gap confirmed (Steps 495-498).	Not killed. Action space limitation confirmed (Steps 495-498). Requires 69-action expansion for FT09.

Family	Experiments	Navigation result	Kill reason
kd-tree	1	0/N. Step 452.	Node splits destroy edges (persistence violated, U3). Immediate kill.
Cellular automata	3	0/N. Steps 449-451.	Degenerate mapping: single attractor state, no cell diversity. Immediate kill.
LLM	1	0/1. Step 462.	Action collapse: 100% ACTION1, no exploration. Preliminary result (n=1).
L2 k-means	~9	LS20: 5/10 at 50K (Step 475), comparable to LSH. FT09: 3/3 (Step 503). VC33: 3/3 (Step 505).	Not killed. Active family. Zone discovery + argmin is general mechanism.
Bloom filter	1	0/10. Step 494.	No edge memory: observation→cell without edge state = near-random walk. Confirms graph mechanism is load-bearing.
SplitTree	5	LS20: 3/3 L1 (Step 537, combined fix: edge transfer + thresh=64). Chain: KILLED (Step 538, deterministic splitting fragments the graph across domains).	Self-partitioning binary tree from transition divergence. Navigates L1 but fully deterministic splitting creates fragile domain separation. 11th distinct family.
Recode	3	LS20: **5/5 L1** (Step 542) — best non-codebook result. Chain: 2/3 L1 with per-domain centering (Step 546).	Not killed. LSH k=16 + passive self-refinement from transition statistics. First substrate where the mapping self-modifies AND navigation succeeds reliably. See Section 5.4.5.

Cross-family findings: - Local continuity (U20) is the strongest kill criterion: 4 families fail

cleanly on this axis (grid graph, CA, reservoir, kd-tree). - centered_enc is load-bearing in 2 families (codebook, LSH) for different failure modes — confirmed universal (U16). - Graph + edge-count argmin is the constant across all winning navigation configurations. No family has won without it. However, random action selection achieves 10/20 L1 at 50K vs argmin's 13/20 (Step 594, p=0.26, NS). The advantage is in SPEED of discovery, not exclusive access. **Proposition 8 (cover time scaling, REVISED by Steps 597/600):** Argmin is a weighted random walk with anti-pheromone weighting. The advantage depends on the budget/graph-size ratio, not graph size alone. At K=16/10K steps, random BEATS argmin (2/5 vs 0/5, Step 597) because the graph is too sparse for reliable visit counts — argmin's few non-zero entries mislead rather than guide. At K=12/10K, argmin leads (3/5 vs 2/5). At K=8/10K, tied (2/5 vs 2/5). There is an optimal K for a given step budget. **Step 600 quantifies the target:** L1 requires 97% cell coverage (186/191 cells, avg degree 3.35/4). Navigation is a coverage threshold problem — the agent must visit nearly all reachable states, not discover the exit early. This explains why argmin helps at the right K (faster coverage) and hurts at wrong K (sparse counts create false gradients). - **Cross-game detection (Steps 575-576):** The mode map + isolated connected-component pipeline generalizes across all 3 ARC-AGI-3 games without game-specific tuning for detection. FT09: 5/5 via LSH k=12 alone (avg 5 cells — trivially solvable by action coverage). VC33: 5/5 via mode map + isolated CC, which autonomously discovers the two magic pixels at (62,26) and (62,34) that Step 505 found with prescribed zone positions. One detection mechanism, three games. - **Constraint validation (Steps 573-574):** U26 (self-labels compound errors) CHALLENGED: LSH k=16 achieves 36.2% test accuracy on P-MNIST with self-labels, 4x above codebook's 9.8%. The self-label failure was codebook-specific (NN-voting on cosine centroids). U19 (dynamics $\neq$ features) PARTIALLY CHALLENGED: LSH k=12 at raw 64\$×\$64$achievesL1reliablyonLS20(1191-$
$1418completionsperseed).Dynamicsalonearesufficientfor L1(single-touchexploration), insufficientfor L2+$
$(sequencingrequirestask-relevantfeatures).-$**Noisy TV is universal:**$Entropy-seeking(Absorbsubstrate, Ste$
$541)drivestheagentintolethalstatesbecausedeathtransitionshavemaximumirreducibleentropy(Burdaetal.2019).$
$entropy-drivenactionselection+BFSrouting)isthe12thtestedarchitecture---KILLEDfornoisyTV.Notcount$
$Classification:$ **R1-compliant classification is unsolved.**$Step432(codebook)achieves94.48\%andStep444b(gra$
$--butbothreceiveground-truthlabelsoneverytrainingstep(supervisedNNC).Withoutexternallabels, accuracydr$
$16achievesNMI = 0.599onCIFAR-100(Step547)---theencodinghasclasssignal, butnomechanismextractsitw$

5.4 The Chain Benchmark

The experiments in Sections 5.1-5.3 test single benchmarks in isolation. The real test is the **chain**: a sequence of heterogeneous benchmarks with one continuous state and no reset.

```
CIFAR-100 → Atari → ARC-AGI-3 → CIFAR-100
```

Each benchmark tests a different capability that specialized systems win: - **CIFAR-100**: classification (DNNs/LLMs win). 100 classes, 32×32 RGB images. R1 prohibits external labels — the substrate must classify by its own dynamics. - **Atari**: navigation/control (RL wins). Reward-driven sequential decision-making. R1 prohibits external reward — the substrate must navigate by exploration. - **ARC-AGI-3**: reasoning (humans win). Interactive games requiring perception, action, and adaptation. Tested in Sections 5.1-5.3.

What the chain tests that single benchmarks don't: 1. **U3 under task switching:** Does the growth-only state (nodes, edges) from CIFAR-100 survive ARC-AGI-3? Or does ARC-AGI-3's

state contaminate CIFAR-100's representations? 2. **R4 across benchmarks:** After ARC-AGI-3 modifies the state, does CIFAR-100 performance degrade? R4 requires that modifications are tested against prior capability. 3. **U11 in sequence:** The substrate must classify (argmax-like) AND navigate (argmin-like) with one state, one mechanism. The chain forces both in sequence, not in parallel.

Protocol: 5-minute phases per benchmark. 1-pass CIFAR (10K images), 5-min per ARC game, 1-pass CIFAR return. One continuous agent state, no reset between phases. Metrics: CIFAR accuracy before/after (forgetting delta), ARC levels reached, total actions to first level completion.

Baselines from literature (reviewer gap #4): No existing method has been evaluated on this exact protocol (cross-domain chain with no reset). For the classification phase, published split-CIFAR-100 results provide context: naive sequential ~10-20%, EWC (Kirkpatrick et al. 2017) ~20-35%, replay buffer (50/class) ~30-45%, DER++ (Buzzega et al. 2020) ~45-55% average accuracy (van de Ven & Tolias, 2024). All exhibit measurable forgetting. The substrate achieves zero forgetting by construction (growth-only state, U3) — no parameter overwriting occurs. Direct comparison is limited by benchmark difference (substrate tested on P-MNIST, not CIFAR-100). The navigation phases have no direct baseline — RL methods require reward (violating R1) and CL methods don't address navigation.

5.4.1 Negative Transfer and Dynamic Growth (Steps 506-508) Frozen centroids from CIFAR break ARC navigation (Step 506: 0/1 at 50K vs 5/10 baseline). This is negative transfer (Rosenstein et al. 2005): CIFAR-trained centroids impose a partition hostile to game frame navigation. ARC frames map to 99/300 CIFAR centroids — the wrong topology.

Dynamic centroid growth fixes this (Step 507: WIN@11170). When new centroids spawn for observations distant from existing ones (L2 > 0.3), CIFAR and ARC centroids naturally separate: CIFAR images have L2 mean=4.3, ARC frames L2 mean=0.5. Zero cross-task interference (CIFAR accuracy delta = -0.05pp).

The full chain passes (Step 508): LS20 WIN@11170, FT09 WIN@8075, VC33 WIN@11, CIFAR delta=-0.01pp. FT09 reuses LS20 centroids almost entirely (2 new centroids spawned). Domain separation is automatic.

5.4.2 The Threshold Tension (Steps 509-513) The encoding (avgpool16 + centered) contains class signal: NMI=0.42 at threshold=3.0 with 2701 centroids (Step 512). NMI climbs monotonically with centroid count (Step 510). But the spawn threshold is incompatible across domains: CIFAR needs threshold ≥ 3.0 for meaningful clustering; ARC needs threshold ≤ 0.5 for navigation. One fixed threshold cannot serve both.

Domain-adaptive threshold (Step 513) auto-calibrates for ARC (median=0.308, matching optimal fixed) but fails for CIFAR — local density in sparse 256D does not reflect class structure. The encoding, not the threshold mechanism, is the bottleneck for classification.

5.4.3 Cross-Family Chain Replication (Steps 515-525) **Negative transfer is universal** (Step 515): K-means with CIFAR-fitted centroids fails on LS20 (3/300 cells) — same collapse as codebook Step 506. Not mechanism-specific.

LSH chain corrected (Steps 516, 523): Single-seed Step 516 (WIN@1116) was an outlier. Multi-seed Step 523 reveals LS20 0/3 — CIFAR edges on actions 0-3 contaminate LS20's 4 directional

actions. Action-scope isolation works only for expanded action spaces (FT09: 69 actions, 3/3; VC33: 3 zones, 3/3) but not for baseline LS20.

Frozen k-means collapses (Step 522): All FT09/VC33 frames map to 1/300 centroids. FT09 3/3 is round-robin exploitability, not transfer.

5.4.4 Algorithm Invariance (Steps 521, 524, 525) **Prior work:** Bisimulation relations (Givan et al., 2003) formalize when two MDP states are behaviorally equivalent — states with the same reward and transition structure over bisimilar successors. MDP homomorphisms (Ravindran & Barto, 2004) extend this to structure-preserving maps between MDPs. State abstraction theory (Abel et al., 2017) characterizes which abstractions preserve near-optimal policies. All concern equivalence of STATES. Our finding is about equivalence of REPRESENTATIONS of the same state — a simpler, more specific result.

Proposition 3 (Representation Invariance): Let $R : \mathcal{H} \to \mathbb{R}^{|A|}$ be a representation mapping transition history $\mathcal{H}$ to a per-action summary vector, and let $g(s) = \text{argmin}_a R(\mathcal{H})_a$. If R is *count-monotone* — i.e., $N(s,a) > N(s,a') \Rightarrow R(\mathcal{H})_a > R(\mathcal{H})_{a'}$ where $N(s,a)$ is the visit count — then g produces identical action sequences regardless of the specific form of R.

Proof: Argmin depends only on the ordering of $R(\mathcal{H})_a$ across actions. Count-monotonicity preserves this ordering. Any two count-monotone representations select the same action. □

Empirical confirmation. Four representations tested on LS20:

Representation	Step	Result	Why count-monotone
LSH graph (edge dict)	459	6/10	$R_a = \sum_n E(s,a,n)$ — direct count
Hebbian weights (matrix W)	524	5/5 (1 trajectory)	$R_a = (W^T x)_a$ — accumulated weight $\propto$ frequency when x is deterministic per state
Markov tensor (T[c,a,c'])	525	8/10	$R_a = \sum_j T[s,a,j]$ — transition total = count
N-gram (history buffer)	521	4/5 (N=20)	Frequency estimate from recent window — converges to count ratios

All converge to **argmin over visit frequency**. Score variations (6/10 vs 8/10) are due to hash randomness and budget, not algorithmic differences.

Relationship to prior work: The proposition itself is mathematically trivial — argmin over orderings is order-invariant. The contribution is empirical: confirming that four architecturally distinct representations ARE count-monotone in practice, and that no representation bias introduces a qualitatively different algorithm. This is NOT bisimulation (which equates states) nor MDP homomorphism (which equates MDPs). It is a sufficient-statistic result: the visit-count ordering is a sufficient statistic for argmin action selection, and all tested representations preserve it.

Implication: The search space for new action-selection mechanisms is constrained. Any count-monotone representation converges to argmin. New representations are unlikely to produce new algorithms unless they introduce a qualitatively different selection rule (not argmin over accumulated state). Combined with Section 4.5 (argmin is robust to noisy TV), this means: the action-selection problem is solved for navigation — argmin is both optimal among tested strategies and invariant across representations. The open problem is self-observation (Theorem 2), not action selection.

5.4.5 Self-Refinement and Navigation Reliability (Steps 534-546, Recode) Recode is LSH k=16 with passive self-refinement: when a cell produces inconsistent transition outcomes (high entropy in $G[(n, a)]$), the substrate learns a separating hyperplane from the mean difference between the two most frequent transition patterns. This splits the cell into two children, each mapping to a distinct transition regime. Refinement is conservative: max 3 splits per 5000 steps, MIN_OBS=8 per transition pair.

Results:

Configuration	Result	Cells	Steps
LSH k=16 (no refinement)	3/3 L1 at 200K (Step 531)	1149	200K
Recode k=16 (with refinement)	**5/5 L1** at 500K (Step 542)	1267	500K
Recode chain (per-domain centering)	2/3 L1 (Step 546)	—	500K
Recode chain (global centering)	0/5 (Step 543)	—	—
LSH k=16 (no centering)	0/5 (Step 544)	62	—

Analysis: The 5/5 vs 3/3 improvement is confounded with budget (500K vs 200K). Self-refinement adds ~10% cells (1267 vs 1149) — modest. The dominant factor in reachable set expansion is k (partition granularity), not refinement. However, Recode is the first substrate where the observation→cell mapping self-modifies AND navigation succeeds reliably. This is partial R3 compliance: the mapping (a component of F) changes based on the state (transition statistics). It is NOT full R3 — the refinement algorithm itself is fixed.

Centering tension resolved (Steps 543-546): Centered encoding ($x - \bar{x}$) is required for navigation (U16: without it, 62 cells, 0/5). But global centering kills domain separation in the chain — CIFAR and LS20 frames hash to shared nodes (Step 543). **Per-domain centering** resets the running mean on domain switch (on_reset signal). Result: 2/3 L1 on the chain, with s0 navigating FASTER than clean (L1@12201 vs 29691). R1-compliant: on_reset is a game event, not an external domain label. The 2/3 vs 5/5 reliability gap on the chain remains unexplained.

Classification (Step 547): Recode's self-refinement is inert on classification — single-pass image presentation never triggers refinement (MIN_OBS=8 per node-action pair is never met with ~1.4 images per node). NMI=0.599 is pure k=16 LSH. Self-refinement is a navigation mechanism, not a general learning mechanism.

R6 diagnostic (Step 548): Does self-refinement satisfy R6 (irredundancy) after exploration saturates? Prediction: <10% of post-saturation refinements change argmin action. **Result: 89.5% — prediction inverted.** Mechanism: when a node splits, one child inherits a fresh edge table; argmin defaults to the least-visited action (typically action 0). Since the pre-split action is rarely action 0 in well-explored cells, the split forces fresh exploration from that partition. 357 total refinements over 599K steps, 304 changed action selection. R6 is satisfied in the narrow sense: each refinement is behaviorally irredundant. Whether this produces new REACHABLE states (not just re-exploration of the same frontier) is tested by Step 549 (trajectory divergence).

R6 and trajectory divergence (Step 549): Recode vs plain LSH (same hash, same seed): Jaccard overlap declines from 0.951 (50K) to 0.798 (300K). But the divergence is entirely partition-based, not frontier-based: at every checkpoint, the number of LSH-unique cells exactly equals the

number of Recode splits ($|unique_{LSH}| = |sp_{Recode}|$). Each split retires a parent from Recode's live set; that parent remains in LSH's set. Recode-unique cells are children of splits — finer partitions of the same spatial region, not new areas of the game world. Recode ends with FEWER live cells (991 vs 1115): trading breadth for depth.

Combined interpretation (Steps 548-549): ℓ_π self-modification satisfies R6 behaviorally (89.5% of splits change argmin) but not frontierally (no new game states reached). The 5/5 vs 3/3 reliability improvement comes from finer state discrimination within the same explored frontier — the agent distinguishes states that were previously confused, enabling more reliable navigation. This is a genuine improvement but does not address Theorem 2's requirement for self-observation after saturation: partition refinement cannot discover states BEYOND the original hash partition's reachable set. ℓ_π is necessary but not sufficient; ℓ_F remains the open question.

Relationship to DESOM (Forest et al. 2021): Deep Embedded Self-Organizing Maps jointly train an autoencoder and a SOM layer, learning representations that are "SOM-friendly" — the encoding adapts to serve the topology-preserving mechanism. Recode is a gradient-free analog: the hash refinement adapts the encoding to serve the argmin mechanism. Both modify π to improve the downstream computation. The difference: DESOM uses backpropagation through a loss function (violating R1), while Recode uses entropy-based splitting from transition statistics (R1-compliant). Step 589 (pending) tests whether this R1-compliant ℓ_π provides a statistically significant advantage over fixed ℓ_0 at 20 seeds.

Non-codebook experiment count: ~177 (vs ~435 codebook). Ongoing scale-up.

6. Degrees of Freedom

The formalization identifies what the constraints REQUIRE but also what they leave UNDETERMINED. These degrees of freedom define the experiment space for the next phase.

#	Degree of Freedom	What's constrained	What's free	Source
1	Growth function ϕ	Must be monotonic, unbounded (U17)	What grows — nodes, edges, meta-state, or something else	Sec 3.2
2	Oscillation period	System cannot converge (Thm 1)	How quickly growth disrupts the dominant mode	Sec 4.1
3	Meta-rule F	Must be non-constant (R3), time-invariant (U1), minimal (R6)	The specific form of compare-select-store	Sec 3.2

#	Degree of Freedom	What's constrained	What's free	Source
4	Growth-rule coupling	Growth changes the update rule (R3)	Local vs global: does a new node change nearby rules or all rules?	Sec 3.3
5	Metric d_X on observations	Must make π Lipschitz (U20); can refine but not rearrange (T3)	Which metric: L2, cosine, learned, adaptive	Sec 3.3
6	Partition resolution	Nodes must be well-separated (Prop 2)	How many nodes: determined by spawn threshold vs observation distribution	Sec 3.3
7	Growth mechanism	Structure only grows (U3), without bound (U17)	Spawn-on-novelty vs implicit edge accumulation vs other	Sec 3.3
8	Self-observation target	Must extract irredundant structure from s (Thm 2)	What to extract: temporal patterns, graph properties, meta-state	Sec 4.3
9	Self-observation $\rightarrow$ rule modification	New structure must change f (R3)	How: does detecting stuckness change exploration? Does cycle detection trigger avoidance?	Sec 4.3
10	Fixed-point structure	Conjectured to exist (Sec 4.4)	What invariant is preserved under self-observation	Sec 4.4
11	Exploration/exploitation resolution	$g_{nav} \neq g_{class}$ (U11), no mode switch (U1)	State-dependent behavior: what state feature triggers the switch?	Sec 3.4

#	Degree of Freedom	What's constrained	What's free	Source
12	Self-modification level	Must be $\geq \ell_\pi$ for R3 compliance (Sec 3.2)	How deep: ℓ_π (Recode) vs ℓ_F (full self-reference). Which component of F adapts, and by what trigger?	Sec 3.2

The central experimental question (DoF 8-9, 12): What specific self-observation mechanism produces irredundant growth after exploration saturates, and at what self-modification level? Recode shows ℓ_π is achievable; whether ℓ_F is necessary or achievable is the open question.

7. Discussion

7.1 Pairwise Consistency Audit

All formalized constraints were checked for mutual consistency. Identified tensions and their resolutions:

Tension	Constraints	Status
T1: Stationarity vs self-modification	U7 + R3	Open — U7 needs reformulation as instantaneous, not asymptotic
T2: Weak vs strong R3	R3 interpretation	**Resolved** — self-modification hierarchy (Sec 3.2). ℓ_π is partial, ℓ_F is full.
T3: Continuity vs self-modification	U20 + R3	Constrained — metric can refine but not rearrange topology
T4: Never delete vs no redundancy	U3 + R6	**Resolved** — irredundant growth (Sec 4.2). Every new component covers unique territory.
T5: Infinite growth vs finite environment	U17 + finite X	**Resolved** — Theorem 2. Growth shifts to state-derived components.
T6: Navigation vs classification	U11 + U24 + U1	Open — state-dependent behavior (DoF 11). No implementation yet.

Tension	Constraints	Status
T7: Self-observation vs noisy TV	Theorem 2 + Sec 4.5	**Sharpened** — Proposition 13 (eigenform inertness) + Proposition 15 (perception-action decoupling): self-observation must target π (observation mapping), not g (action selection). Recode achieves this via ℓ_π. Forward prediction remains required for L2.
T8: Centering vs domain separation	U16 + chain benchmark	**Resolved** — per-domain centering (Step 546). R1-compliant via on_reset.

No undiscovered contradictions. All tensions are either resolved (T2, T4, T5, T8), constrained (T3), or identified as open questions (T1, T6, T7). The constraint system is internally consistent.

7.2 What is proven

1. The system has no fixed point (Theorem 1). Self-modification is necessary.
2. In finite environments, the system must process its own state (Theorem 2). Self-observation is required.
3. R3 and R5 are simultaneously satisfiable if and only if the ground truth is strictly environmental (Theorem 3, System Boundary Theorem). The feasible region is not provably empty — the question is empirical.
4. The current graph + argmin mechanism exits the feasible region after exploration saturates.
5. Argmin over visit counts is robust to noisy TV and representation-invariant (Section 4.5, Proposition 3). Action selection for navigation is solved.
6. Self-modification of the encoding (ℓ_π) is achievable and improves navigation reliability (Recode, Step 542).
7. Permanent soft death penalty (ℓ_1) does NOT improve navigation success rate at 50K steps: 13/20 vs 13/20 (Step 584, p=0.63). However, it accelerates early discovery: 9/20 vs 6/20 at 10K (Fisher p=0.33). The mechanism is a speed improvement, not a success improvement — argmin catches up by 30K. Cross-game test (Step 585): NEUTRAL on VC33 — death penalty is navigation-specific, not universal.

8. R1 (no external objectives) costs ~1 level (~6%) at the ARC-AGI-3 competition frontier. The 16 levels solved via source analysis are R3's SPECIFICATION — 16 concrete test cases — not evidence that R3 is nearly satisfied (Proposition 9, competition data).

9. The gap from ℓ_1 to ℓ_F is recording vs predicting ground truth events (Proposition 10). 577d pixel statistics navigate to the WRONG cells; 581d death penalties improve navigation via retrospective marking. Prospective prediction requires features that correlate with ground truth — which R1 prohibits optimizing for directly.

10. The frozen frame and navigation capability are structurally coupled (Proposition 11, R3_AUDIT.md). The 5 unjustified elements that enable navigation are the same elements R3 requires the system to self-modify. R3 requires exploring modifications to load-bearing components. **Dissolved by Proposition 14 corollary:** the coupling is between abstract structure (entailed, fixed) and concrete implementation (variable, discoverable). R3 searches implementations, not operations.

11. The minimal frozen frame of any self-modifying substrate is the interpreter (compare-select-store) + the ground truth (R5) (Proposition 12). ℓ_F as written is impossible for computable systems — the interpreter cannot rewrite itself without infinite regress (Von Neumann, Kleene, Schmidhuber). But ℓ_F is achievable IN EFFECT via expressive ℓ_1: if the state space encodes operations the interpreter executes, the system's behavior is indistinguishable from ℓ_F. Closure to efficient causation IS computable (Mossio & Longo 2009) — the barrier is expressiveness, not computability.

12. Self-observation via the eigenform $F(s)(\text{enc}(s))$ converges to a fixed point and is inert for navigation (Proposition 13, Steps 620-629). Self-observation is necessary (Theorem 2) but not sufficient for L2+. The gap is informational: the graph stores the past; L2 requires predicting the future. Introspection $\neq$ foresight.

7.3 What is conjectured

12. The minimal self-observing substrate is an eigenform of F: F applied to its own state reproduces its dynamics (Section 4.4).
13. The oscillation between U7 (convergence) and U17 (growth) is a limit cycle, not chaos.
14. Selection pressure on a population of encodings (GRN architecture) may bridge the →_F gap without optimization. Step 607 tested: KILLED (Cov(w,z)=0). Framework sound but requires behaviorally diverse encodings.

7.4 What is open

15. Whether the full feasible region (R1-R6 + all validated U-constraints) is occupied. Theorem 3 shows it is not provably empty; a witness or tighter impossibility result is needed.
16. What self-observation mechanism satisfies R6 (irredundancy) while avoiding noisy TV traps (T7). The eigenform mechanism (Section 4.4) is a concrete candidate; Zenil's degeneration constraint (2026) predicts interleaving with environment observation is required.
17. How to resolve U11 + U24 + U1 (incompatible tasks, no mode switch) in a single system (T6). Recent work (Hao et al. 2025, "Beyond the Exploration-Exploitation Trade-off," arXiv 2509.23808) argues the tradeoff is an artifact of measurement level — in hidden-state space,

exploration and exploitation decouple. The eigenform mechanism (Section 4.4) may dissolve U24 similarly: at the meta-cell level, argmin naturally produces exploitation (follow meta-cell recommendations) or exploration (least-tried action) depending on state richness, without mode switching.

18. R1-compliant classification — no substrate has achieved above-chance accuracy without external labels.
19. **Interpreter entailment (Proposition 14).** The abstract interpreter is entailed by R1-R6. The concrete interpreter is not. The computability question is resolved: Mossio & Longo (2009) showed closure to efficient causation IS computable in reflexive domains. The remaining question is expressiveness: can a specific interpreter + state space combination encode enough operations to discover its own implementations? Rosen's -map must be postulated (Letelier et al. 2006); our interpreter is analogously postulated. Whether the constructive gap is bridgeable — transitioning from externally imposed constraints (R1-R6) to internally maintained constraint closure (Montévil & Mossio 2015) — connects to the _1 → _F bridge and remains the central open problem.
20. **Targeted -refinement without foreknowledge — RESOLVED (Proposition 16).** Transition-inconsistency refinement (Step 674, 17/20 L1 at 25s) achieves R1-compliant targeted π-refinement. Cells with $I(n) \geq 2$ (inconsistent transitions) are refined with finer hash. The exit cell naturally has the highest inconsistency because it is where hidden-state conjunctions create transition variability. The mechanism discovers task-critical cells without foreknowledge. Confirmed across 20 seeds (Step 690) and 2 games (FT09 5/5, Step 680). **Remaining open:** why the binary criterion (any inconsistency) outperforms ranked/capped variants, and whether the approach generalizes beyond navigation to classification or other tasks.
21. Whether the state space of compare-select-store is expressive enough to encode arbitrary self-modifications as data (the expressiveness question from Proposition 12). SUBLEQ (subtract-and-branch-if--0) is Turing complete with one instruction and unbounded memory. Compare-select-store maps structurally: COMPARE → subtract, SELECT → branch, STORE → write. But our specific COMPARE (LSH hash) is a fixed random projection — not an arbitrary function. The system's expressiveness is bounded by the comparison function's expressiveness. Whether the eigenform mechanism (hashing action counts → meta-comparison) iterates toward Turing-complete comparison is the core open question. If yes, R3 is satisfiable with the current interpreter. If no, the comparison function must be extended or made adaptive ($\ell_\pi \to \ell_F$ in effect).

7.5 The Level 2 Problem

Steps 548-562 systematically eliminated candidate explanations for the Level 2 failure. The state graph at 500K steps contains 942 live cells in a single connected component, with the agent circulating through a 364-node active set (29% of nodes). 134 deterministic frontier edges remain unexploited (Step 550). Increasing partition resolution to k=20 (1749 cells) does not reach L2 (Step 551). Aggressive splitting disrupts the attractor (4970 active cells) without reaching L2 (Step 554). Extended budget produces no improvement — the active set plateaus at ~5000 (Step 555).

The root cause is the game's energy mechanic: each life lasts 43 steps (3 lives = 129 steps per episode). Level 1 is reachable by random walk within this budget. Level 2 introduces energy palette sprites ("iri") that refill the step counter — the agent must navigate TO them before

running out of energy, then proceed to the exit (Steps 556-557). Rudakov et al. (2025) solved L2 in 4000 actions using connected-component segmentation with visual salience prioritization.

The encoding (avgpool$_{16}$: 64\$×\$64 → 16\$×\$16 = 256$D)istoolossytoresolve3-pixelsprites.AnR1-compliantself-observationsignalexists---frame-diffisperfectlybimodalwithagapat0.082separatingmovem--andusingittoskipwastedactionsreachesL12xfaster(Step559).ButthissignaldetectswhethertheagentMOVED directednavigation(connected-componentsegmentationwithoutsaliencefiltering)killsL1entirely-- - 97.7%ofactionschasewallsandfloor(Step561).ThenoisyTVproblemappliestoobjects : withoutdiscriminatinginteractivefromdecorativeelements, object-chasingisworsethanargmin.$$$

7.6 Honest assessment

The feasible region for Level 1 navigation is occupied — graph + argmin + correct action decomposition satisfies R1, R2, U1, U3, U17, U20 and solves all three games. However, argmin's advantage over random action selection is a speed improvement, not exclusive access: 13/20 vs 10/20 at 50K (Step 594, p=0.26, NS). LS20 Level 1 is within random-walk reach at sufficient budget. Level 2 (mgu completion) is achieved via a 12-component prescribed pipeline: mode map, isolated connected-component detection, level-aware reset, multi-episode accumulation, dead reckoning, state estimation from pixel diffs, and sequenced visitation (Step 572j, L2=5/5 at avg 4804 steps). The pipeline is R1-compliant in its detection components (no external labels) but game-specific in its state estimation (pixel regions and visit ordering are prescribed from source code analysis).

The gap to R3 is precisely enumerated at two levels:

At the pipeline level (L2+): 12 design choices that the substrate cannot currently self-discover. Four are general techniques (mode map, isolated CC, level-aware reset, multi-episode accumulation). Eight are game-specific engineering (position tracking, state estimation, sequencing, threshold tuning). A substrate satisfying R3 would need to discover all 12 from interaction alone — including recognizing that the game has hidden state variables gating progression (the POMDP structure).

At the substrate level (L1): The R3 audit (R3_AUDIT.md) reveals a deeper problem. The navigating substrate (process_novelty) has 5-7 unjustified elements, and the 5 that enable navigation (class-restricted spawn, class-restricted attract, argmin class scoring, top-K scoring, seeding protocol) ARE the frozen frame. Removing any of them produces SelfRef, which does not navigate. The frozen frame and navigation capability are **coupled through the class structure**: the U elements that R3 requires the system to self-modify are the same elements that make navigation work. This is not an engineering problem — it is a structural tension between R3 (self-modify operations) and the fact that the operations being modified are load-bearing. The system cannot safely explore modifications to its own class structure because every modification kills navigation.

This parallels the U16 centering finding (Proposition 7): some frozen elements are so load-bearing that self-modification of them is functionally suicidal. The system cannot learn to remove centering because every removal kills it. Similarly, the system cannot learn to restructure its class scoring because every restructuring kills it. R3 requires exploring modifications to elements whose modification is lethal — a much harder problem than discovering new encodings or action spaces.

No current substrate family approaches this. The full R1-R6 region remains unoccupied.

The R3-R5 Tension (Proposition 4) R5 requires one fixed ground truth. R3 requires every aspect of computation to be self-modifiable. The tension is: if evaluation criteria are fixed (R5), then at minimum the system's notion of "good" is prescribed — violating R3.

Resolution: R5's ground truth is *environmental feedback* (game score, death events, level transitions). It is input the substrate reads, not computation the substrate performs. R3 applies to the substrate's *response* to ground truth, not to the ground truth signal itself. The resolution holds if and only if: (a) the ground truth is strictly environmental (no designer-specified reward shaping), and (b) the substrate's interpretation of ground truth is itself self-modifiable.

581d under the hierarchy: Step 581d (permanent soft death penalty, 4/5 vs argmin 3/5) is the closest empirical approach to R3. But it is ℓ_1, not ℓ_π: the system self-selects WHICH edges to

penalize (death edges — data-driven), but the penalty VALUE (fixed constant) and DURATION (permanent) are designer-prescribed. The response to ground truth is partially self-modified (placement), partially hardcoded (magnitude, persistence). Full R3 requires ℓ_F: the system modifies its own rule for how to respond to death. The gap from ℓ_1 (581d) to ℓ_F is two qualitative levels — this is the central open problem.

Theorem 3: The System Boundary Theorem **Statement:** R3 (full self-modification) and R5 (fixed ground truth) are simultaneously satisfiable if and only if the ground truth g is strictly environmental — i.e., $g \notin F$.

Proof sketch:

($\Rightarrow$) Suppose g is a component of F. R3 requires every component of F to be self-modifiable. If g is self-modifiable, the system can alter its evaluation criterion, violating R5. If g is protected from modification, that protection mechanism is a non-self-modifiable component of F, violating R3. Contradiction in both cases.

($\Leftarrow$) Suppose g is strictly environmental (determined by the environment, not by any component of F). Then R3's requirement that every component of F be self-modifiable does not apply to g, since $g \notin F$. R5 holds because g is fixed by the environment. No contradiction.

Implication: The boundary of self-modification is precisely the system-environment interface. Everything inside the system boundary (encoding, update, selection, meta-rules) must be self-modifiable for R3. Everything outside (ground truth signal) must be fixed for R5. The theorem constrains substrate design: the ground truth MUST be environmental feedback (game death, level transitions, external reward), never an internal evaluation function. Any substrate with an internal fitness function violates R3 or R5.

Proposition 6: Self-Modification Level Determines Speed vs Reachability **Statement:** ℓ_1 self-modification (data-driven placement of prescribed operations) accelerates exploration but does not expand the reachable set. Only ℓ_π self-modification (data-driven encoding change) expands reachability.

Evidence: - Step 584 (20 seeds, 50K): Soft penalty (SP) and argmin (AM) converge to 13/20 at 50K, but SP leads 9/6 at 10K. The penalty accelerates discovery without expanding the graph. - Step 542 (Recode, 5/5): Self-modification of π (hash refinement from transition statistics) expands the graph from 440 to 1267 cells and improves success from 3/3 to 5/5. The encoding change creates new nodes the agent can visit. - Step 587: Death-count penalty DC(1) solves at 2338 steps vs SP at 7749 — faster still, but on the same graph.

Interpretation: The reachable set is determined by the partition geometry (π). ℓ_1 operations (penalty, boost) change how the agent traverses the existing graph. ℓ_π operations (Recode) change what the graph IS. Speed vs reachability maps exactly to ℓ_1 vs ℓ_π in the self-modification hierarchy.

Testable prediction (FALSIFIED by Step 589): Proposition 6 predicted Recode (ℓ_π) would outperform LSH (ℓ_0) at 20 seeds. Result: Recode(K=16) 18/20 = LSH(K=16) 18/20 > LSH(K=12) 13/20. The advantage over K=12 is entirely explained by having 16 hash bits, not by adaptive splitting. ℓ_π self-modification adds nothing on top of the K it uses. The hierarchy remains descriptively useful (categorizing mechanisms by what they modify) but is not operationally predictive: more hash bits (a frozen ℓ_0 parameter) achieves the same reachability as adaptive splitting (ℓ_π).

The speed-vs-reachability distinction holds at mid-budget (Recode leads at 30-40K checkpoints, $p<0.05$) but collapses at 50K.

Proposition 9: The R1 Tax **Statement:** R1 (no external objectives) costs approximately one level (~6%) at the ARC-AGI-3 competition frontier. The cost is bounded and non-fatal.

Evidence (competition data as measurement instrument): - StochasticGoose (Tufa Labs, 1st place): CNN + RL, 18 levels across 2 games. Violates R1 (uses reward signal for RL training). Violates R3 (fixed CNN architecture, fixed RL policy update). - Rudakov et al. (dolphin-in-a-coma, 3rd place): Graph-based exploration, 17 levels post-fix (14-19 range across seeds). Satisfies R1 (no reward, no training). Violates R3 (fixed CC segmentation, fixed priority tiers, fixed BFS planning). - Our system: Graph + argmin + source analysis, 16 levels (3 LS20 + 6 FT09 + 7 VC33). Satisfies R1. Violates R3 (prescribed pipelines per game). FT09: all 6 levels, 75 clicks (Step 608b). VC33: all 7 levels, 176 clicks (Step 610). LS20: L1-L3 via mgu pipeline; L4+ blocked at L3 transition.

Interpretation: The gap between R1-violating (18 levels) and R1-satisfying (17 levels) systems is ~1 level. R1 is cheap. But source analysis does not close the R3 gap — **source analysis IS the R3 violation.** Reading game source code to design level-specific pipelines (FT09 color matching, VC33 analytical BFS, LS20 mgu navigation) is precisely what R3 prohibits: a human engineer prescribing the system's operations. The 16 levels we solved by source analysis are not evidence that R3 is nearly satisfied. They are 16 concrete test cases that specify what an R3-compliant substrate must discover autonomously:

- **FT09 (6 levels, 75 clicks):** The substrate must discover color-matching rules from pixel observations alone — that cgj() checks 8 neighbors against bsT sprite target colors.
- **VC33 (7 levels, 176 clicks):** The substrate must discover gel physics, switch conditions, and train tracking — then solve a 2.4M-state BFS without being given the state representation.
- **LS20 (3 levels, mgu pipeline):** The substrate must discover spatial navigation, energy management, and level geometry changes across L1→L2→L3 transitions.

Each additional level we solve via source analysis makes the R3 specification MORE precise, not the R3 gap smaller. The gap is the entire distance from ℓ_0 (fixed operations) to ℓ_F (self-modifying frozen frame). No substrate we have tested crosses this gap. Rudakov et al.'s system (17 levels, no source analysis) uses fixed CC segmentation, priority tiers, and BFS planning — the same class of prescribed operations. Their success at R1 + fixed strategy confirms: **the feasible region for R1 alone is large; the feasible region for R1 + R3 is where the real constraint lies.** The 16 levels enumerate what lives inside the R3 gap.

Caveat: Competition data is biased — top teams optimize for competition metrics, not R1-R6 compliance. The comparison is informative, not definitive. N=3 teams at the frontier.

Proposition 10: The Feature-Ground Truth Coupling Barrier **Prior work:** Jin et al. (2020, "Reward-Free Exploration") prove that polynomial exploration suffices to learn a representation from which near-optimal policies can be computed for ANY downstream reward function. However, their framework assumes reward IS eventually given — the exploration phase is reward-free, but the planning phase uses reward. Our R1 prohibits external reward entirely. R5 provides environmental ground truth (death, level transitions) as the only feedback signal.

Statement: The gap from ℓ_1 to ℓ_F in the self-modification hierarchy is the gap between RECORDING ground truth events and PREDICTING them.

Evidence: - Step 577d: pixel statistics (mode, mean, variance, diff, min) used as navigation features visit MORE cells than argmin (higher exploration rate) but the WRONG cells (0/5 L1 across 390 evaluation windows $\times$ 5 feature buffers). Visual statistics are not predictive of exit-relevant features. - Step 581d: death events used as post-hoc soft penalties achieve 4/5 vs argmin 3/5. Ground truth feedback improves navigation when applied as ℓ_1 (penalty placement). - Step 589: encoding self-modification (ℓ_π, Recode) provides no advantage over fixed encoding (ℓ_0, LSH) at same K. The system modifies WHERE hyperplanes split, but not WHAT the splits are FOR.

Formalization: Define the ground truth predictor $p : S \to [0, 1]$ as the substrate's estimate of $P(\text{ground truth event}|s)$. At ℓ_1, p is retrospective: ground truth events mark states after they occur. At ℓ_F, p is prospective: the substrate predicts which states WILL produce ground truth events and navigates accordingly. The barrier is: prospective prediction requires features that correlate with ground truth, but R1 prohibits optimizing features FOR ground truth correlation. The features must emerge from the dynamics alone.

Implication: The GRN architecture (below) may address this: if multiple encodings $\{\pi_i\}$ compete, and environmental ground truth selects the winner, then the surviving π_i is the one whose features happen to correlate with ground truth — without any π_i being optimized for it. Selection pressure, not optimization, bridges the gap.

Proposition 11: The Frozen Frame — Capability Coupling **Prior work:** Self-adaptive systems literature (Salehie & Tahvildari, 2009) identifies the challenge that reconfiguring essential components risks disrupting system functionality. No formal framework addresses the specific case where the elements REQUIRING modification for self-improvement are exactly the elements whose modification destroys capability.

Statement: Let $\mathcal{U}(F) = \{u \in F : u \text{ is neither Modified nor Irreducible}\}$ be the set of unjustified frozen elements. For the navigating substrate (process_novelty), $\mathcal{U}(F)$ is load-bearing: for every $u \in \mathcal{U}(F)$, removing u reduces navigation success to 0. R3 requires every element of $\mathcal{U}(F)$ to become Modified. But modifying any element of $\mathcal{U}(F)$ is lethal.

Evidence (R3_AUDIT.md, Rounds A-B): - P6 (top-K class scoring): Remove $\to$ SelfRef (no class structure) $\to$ 0 levels. - P7 (argmin class selection): Remove $\to$ random or argmax $\to$ 0 levels. - P8 (class-restricted spawn): Remove $\to$ global spawn (SelfRef) $\to$ cb=164 vs 20K, 0 levels. - P9 (class-restricted attract): Remove $\to$ global attract (SelfRef) $\to$ 0 levels. - P15 (seeding protocol): Remove $\to$ cold-start degenerate argmin $\to$ 0 levels.

Formalization: Define the **capability coupling** of F as $C(F) = \mathcal{U}(F) \cap \mathcal{L}(F)$, where $\mathcal{L}(F) = \{u \in F : \text{removing } u \text{ kills the ground truth test}\}$ is the set of load-bearing elements. R3 requires $\mathcal{U}(F) = \emptyset$. The coupling $C(F) = \mathcal{U}(F) \cap \mathcal{L}(F)$ measures how much of the unjustified frozen frame IS the capability. For the navigating substrate, $C(F) = \mathcal{U}(F)$ — the coupling is total.

Relationship to prior work: This is a stronger obstacle than Schmidhuber's Gödel machine incompleteness (where the system cannot PROVE certain rewrites are safe). Here the system cannot SURVIVE certain rewrites — not because proof is hard, but because the rewrites are genuinely destructive. The obstacle is physical, not logical.

Implication: R3 cannot be achieved by incremental modification of existing substrates. Removing any U element kills navigation; adding self-modification of U elements adds complexity (violating

U4) without removing the underlying dependency. The R3-compliant substrate, if it exists, must achieve navigation through a DIFFERENT mechanism — one where the load-bearing elements are Irreducible (forced by the constraint map) rather than Unjustified (chosen by the designer).

Population-Level R3: Fixed Rules, Collective Self-Modification The unifying principle. Every natural self-organizing system that achieves complex adaptive behavior does so from fixed individual rules operating through a shared medium. Gene regulatory networks: the genome is fixed, but which genes are expressed changes via environmental signals and mutual inhibition (Oliveri & Davidson, 2008). Quorum sensing: each bacterium has fixed response rules (ℓ_0), but collective gene expression self-modifies when autoinducer concentration crosses a threshold (Waters & Bassler, 2005). Physarum polycephalum: the thickening rule is fixed (ℓ_1), but the tube network self-organizes to solve shortest-path problems (Tero et al., 2010). Turing reaction-diffusion: the equation is fixed, but stable spatial patterns emerge from a homogeneous substrate (Turing, 1952). Ant colony optimization: pheromone response is fixed, but trail networks self-organize to find efficient paths (Dorigo et al., 2000).

None of these systems modify their own rules. All achieve self-organization that LOOKS like ℓ_F from the outside.

Mapping to our substrate. The edge-count graph IS both memory and processor — like Physarum's tube network. Edge counts ARE autoinducers — like quorum sensing. Argmin IS a fixed response rule (ℓ_0) that produces self-modifying exploration at the collective level, because graph topology changes the count distribution which changes the actions.

The R3 architecture this suggests. Instead of one encoding π that modifies itself (Schmidhuber's self-referential approach), maintain multiple encoding candidates $\{\pi_1, \pi_2, \ldots\}$ that mutually inhibit each other. Environmental ground truth (R5) determines which π_i is expressed. The substrate doesn't modify π — it modifies which π is *active*. Whether this achieves genuine ℓ_F or disguised ℓ_1 (the "population" being data, not operations) is an open question.

Mathematical framework. The Price equation (Price, 1970) partitions evolutionary change: $\Delta\bar{z} = \mathrm{Cov}(w, z)/\bar{w} + E(w \cdot \Delta z)/\bar{w}$. The first term is selection (Step 607). The second is transmission/mutation. Step 607 tested selection only: KILLED — Cov(w,z)=0, encodings not behaviorally diverse. The failure is in encoding generation (all LSH encodings at same k produce similar hash distributions), not in the selection framework. The Price equation predicts success requires behaviorally diverse — not just parametrically diverse — encoding candidates.

The sharpened question. R3 may be asking the system to do something no natural self-organizing system does within a single lifetime. Physarum, Turing patterns, and ant colonies all achieve complex adaptation from fixed ℓ_1 rules. In biology, the rules ARE modified — but by evolution, a much slower meta-process operating on a population over generations. R3 at the individual level may require population-level dynamics compressed into a single agent's lifetime. This is the engineering challenge: generating sufficient diversity for selection to operate, within the 5-minute experiment cap.

Proposition 12: The Interpreter Bound Prior work: Von Neumann's universal constructor (1966) separates the CONSTRUCTOR (interpreter) from the DESCRIPTION (data). The constructor reads descriptions and builds machines. The description is copied into new machines. The constructor is the irreducible frozen frame — it cannot be built by a simpler constructor without

infinite regress. Kleene's recursion theorem (1938) guarantees that for any computable function F, a fixed point exists: a program e such that $\varphi_e = \varphi_{F(e)}$. The program can act on its own description without modifying its own interpreter. Schmidhuber's SRWM (Irie & Schlag, 2022) collapses meta-learning levels into one self-referential weight matrix but acknowledges: "the highest level programmer would still have a fixed weight matrix."

Statement: The minimal frozen frame of any self-modifying substrate is the interpreter — the mechanism that reads state and produces behavior. Formally: if F decomposes as $F(s)(x) = \texttt{store}(s, \texttt{select}(s, \texttt{compare}(s, x)))$, then $\{\texttt{compare}, \texttt{select}, \texttt{store}\}$ is irreducible. The state s can encode arbitrary instructions that these operations execute, but the operations themselves cannot be modified from within.

Derivation: Suppose `compare` could be modified by s. Then $F(s_1) \neq F(s_2)$ as operations — which is R3 (the meta-rule is non-constant). But the MECHANISM by which s modifies `compare` is itself a fixed operation — call it `modify_compare`. This operation is now part of the frozen frame. Recursion: can `modify_compare` be modified? Only by another fixed operation. The regress terminates at the outermost interpreter, which is irreducible by construction.

The minimum: Combining with Theorem 3 (R3+R5 satisfiable iff ground truth is environmental), the minimal frozen frame is: **the interpreter (compare-select-store) + the ground truth test (R5)**. Everything below — state, data, descriptions, encodings — is potentially self-modifiable through the interpreter. The eigenform mechanism (Section 4.4) shows how: the state encodes meta-instructions (action count vectors → meta-cells) that the interpreter executes, modifying the substrate's behavior without modifying the interpreter itself. This is ℓ_1 by the hierarchy — but it is the MAXIMUM achievable ℓ for any computable system.

Relationship to prior work: Von Neumann's constructor = our compare-select-store. Kleene's fixed point = our eigenform. Rosen's (M,R) closure = our autopoietic growth (U17). The individual pieces are known; the synthesis — that the interpreter IS the irreducible frozen frame, and R3 is satisfiable iff "self-modification" means "state encodes operations the interpreter executes" rather than "the interpreter rewrites itself" — appears to be our contribution.

Implication: ℓ_F as originally defined (the rule for modifying π itself adapts) is impossible for computable systems — the highest-level rule is always fixed. But ℓ_F can be ACHIEVED IN EFFECT by a sufficiently expressive ℓ_1: if the state space S is rich enough to encode arbitrary comparison operations as data, and the interpreter executes them, then the system's behavior is indistinguishable from ℓ_F despite being formally ℓ_1. The question shifts from "can the system modify its interpreter?" (no) to "is the interpreter + state space expressive enough to simulate any modification?" (the expressiveness question).

7.7 Where the search points

The 720+ experiments, 12 propositions, and 3 theorems converge on a single question: **is compare-select-store with a sufficiently rich state space expressive enough to simulate self-modification?**

The path to this question: 1. R3 requires self-modification of operations (the goal). 2. Proposition 11 shows the current frozen frame IS the navigation capability — you can't modify one without destroying the other (the obstacle). 3. Proposition 12 shows the interpreter (compare-select-store) is irreducible — every self-referential system has a top-level frozen frame (the bound). 4. But Proposition 12 also shows _F is achievable IN EFFECT if the state space encodes operations

the interpreter executes (the opening). 5. The eigenform mechanism (Section 4.4) shows how: $F(s)(\text{enc}(s))$ — the substrate applies its own operations to its own state, building a meta-graph that transfers experience across similar cells (the mechanism). 6. Zenil (2026) predicts this works only when interleaved with environmental observation — pure self-reference degenerates (the constraint). 7. VERL (Hao et al. 2025) suggests the explore/exploit tradeoff dissolves at the right measurement level — the eigenform's meta-cells may provide that level (the bonus).

If the eigenform experiment (Step 617) shows that meta-graph tie-breaking improves navigation, Theorem 2 is validated empirically and the path toward R3 is: build richer state encodings that the fixed interpreter executes. If it fails, the interpreter itself must be extended — and the question becomes what the minimal extension is.

Either way, the 16 levels solved via source analysis (Proposition 9) are the specification: 16 concrete test cases that the resulting substrate must solve autonomously, without prescribed pipelines, without source code analysis, without a human designer.

The contribution is the walls. The substrate — if it exists — lives inside them.

Author Attribution and Disclosure

This research was conducted using a team of LLM agents (Claude Opus and Claude Sonnet) coordinated by the author. Experiments were designed, theory was formalized, and the paper was written with LLM assistance. Experiment scripts were implemented and executed via LLM agents. Strategic direction, the constitutional framework (R1-R6), approval gates, and evaluation of findings for self-deception were provided by the author.

The adversary process (review of each experiment) was conducted by the LLM agents, not by independent human reviewers. The simulated NeurIPS review in Section 7 was generated to stress-test the paper's claims. All experimental code is available in the repository for independent verification.

The agents operated on a single machine (Windows 11, RTX 4090) with experiments run via WSL2. The memory system, mailbox, and coordination infrastructure are documented in the repository.

References

- Abraham, W. C. & Robins, A. (2005). Memory retention — the synaptic stability versus plasticity dilemma. Trends in Neurosciences, 28(2), 73-78.
- Bellemare, M. et al. (2016). Unifying Count-Based Exploration and Intrinsic Motivation. NeurIPS.
- Burda, Y. et al. (2019). Large-Scale Study of Curiosity-Driven Learning. ICLR.
- Fritzke, B. (1995). A Growing Neural Gas Network Learns Topologies. NeurIPS.
- Givan, R., Dean, T. & Greig, M. (2003). Equivalence Notions and Model Minimization in Markov Decision Processes. Artificial Intelligence, 147(1-2), 163-223.
- Graves, A. et al. (2014). Neural Turing Machines. arXiv:1410.5401.
- Jin, C. et al. (2020). Reward-Free Exploration for Reinforcement Learning. ICML.
- Kirsch, L. & Schmidhuber, J. (2022). Self-Referential Meta Learning. ICML.
- Kohonen, T. (1988). Self-Organization and Associative Memory. Springer.
- Maturana, H. & Varela, F. (1972). Autopoiesis and Cognition: The Realization of the Living.
- McCloskey, M. & Cohen, N. J. (1989). Catastrophic interference in connectionist networks. Psychology of Learning and Motivation, 24, 109-165.

- Pathak, D. et al. (2017). Curiosity-driven Exploration by Self-Supervised Prediction. ICML.
- Ravindran, B. & Barto, A. G. (2004). Approximate Homomorphisms: A Framework for Non-Exact Minimization in Markov Decision Processes. ICML Workshop.
- Rosenstein, M. et al. (2005). To Transfer or Not To Transfer. NIPS Workshop on Inductive Transfer.
- Rudakov, E., Shock, J. & Cowley, B. U. (2025). Graph-Based Exploration for ARC-AGI-3 Interactive Reasoning Tasks. arXiv:2512.24156.
- Schmidhuber, J. (2003). Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements. arXiv:cs/0309048.
- Strehl, A. L. & Littman, M. L. (2008). An Analysis of Model-Based Interval Estimation for Markov Decision Processes. JCSS, 74(8), 1309-1331.
- van de Ven, G. M. & Tolias, A. S. (2024). Continual Learning and Catastrophic Forgetting. arXiv:2403.05175.
- Wang, Z. et al. (2019). Characterizing and Avoiding Negative Transfer. CVPR.
- Zenil, H. (2026). On the Limits of Self-Improving in Large Language Models: The Singularity Is Not Near Without Symbolic Model Synthesis. arXiv:2601.05280.
- Kauffman, L. H. (2023). Autopoiesis and Eigenform. Computation, 11(12), 247.
- Tero, A. et al. (2010). Rules for Biologically Inspired Adaptive Network Design. Science, 327(5964), 439-442.
- Rosen, R. (1991). Life Itself: A Comprehensive Inquiry into the Nature, Origin, and Fabrication of Life. Columbia University Press.